


# EN3150 Pattern Recognition
# Convolutional neural networks (CNNs)

Most of the contents are based on

1. C. M. Bishop, Pattern Recognition and Machine Learning.
2. Kevin P. Murphy, Machine Learning: A Probabilistic Perspective.
3. Prof. Svetlana Lazebnik, Prof. Derek Hoiem University of Illinois at Urbana-Champaign.
4. CS231n: Deep Learning for Computer Vision (Stanford) (Dr. Justin Johnson).

---

Presented by M. T. U. Sampath K. Perera,
Department of Electronic and Telecommunication Engineering,
University of Moratuwa.
(sampathk@uom.lk).
Semester 5 – Batch 21.

# Convolutional neural networks (CNNs)

- Why MLP not good structured grid-like data, such as images and sequences?
  - **Lack spatial understanding**
    - Treat input data as a flat vector, ignoring the spatial relationships and structures within the data
  - **Do not capture local patterns**
    - Do not capture local patterns or relationships in data
  - **Cannot recognize patterns regardless of their position**



# Convolutional neural networks (CNNs)

- **Convolutional neural networks** (**CNN**s)
  - Preserve the spatial structure of data  (capture local patterns and relationships)
  - Translation invariant (can recognize patterns regardless of their position)
  - Weight sharing across different regions of the input data
  - Hierarchical feature extraction (well-suited for capturing complex patterns in data)
    - E.g., detect edges, textures, and higher-level features in images

# Convolutional neural networks (CNNs)

- **Convolution**
  - **1d convolution**

$$[f \circledast g](\boldsymbol{z}) = \int_{\mathbb{R}^D} f(\boldsymbol{u}) g(\boldsymbol{z} - \boldsymbol{u}) du$$

  - **Discrete convolution**

| 1 | 2 | 3 | 4 | ⊛ | 5 | 6 | 7 | = | 5 | 16 | 34 | 52 | 45 | 28 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

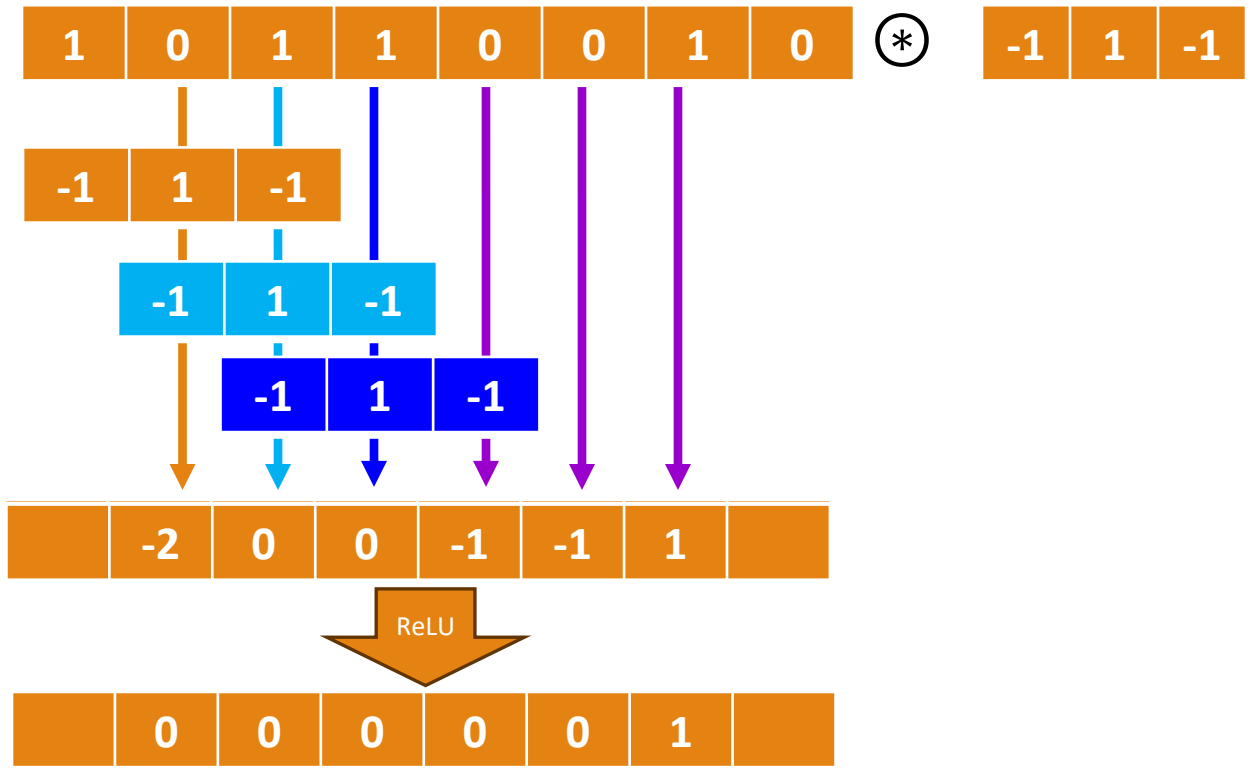
| - | - | 1 | 2 | 3 | 4 | - | - | |
|---|---|---|---|---|---|---|---|---|
| 7 | 6 | 5 | - | - | - | - | - | $z_0 = x_0 w_0 = 5$ |
| - | 7 | 6 | 5 | - | - | - | - | $z_1 = x_0 w_1 + x_1 w_0 = 16$ |
| - | - | 7 | 6 | 5 | - | - | - | $z_2 = x_0 w_2 + x_1 w_1 + x_2 w_0 = 34$ |
| - | - | - | 7 | 6 | 5 | - | - | $z_3 = x_1 w_2 + x_2 w_1 + x_3 w_0 = 52$ |
| - | - | - | - | 7 | 6 | 5 | - | $z_4 = x_2 w_2 + x_3 w_1 = 45$ |
| - | - | - | - | - | 7 | 6 | 5 | $z_5 = x_3 w_2 = 28$ |

$$[w \circledast x](i) = \sum_{u=0}^{L-1} w_u \, x_{i+u}$$

If weights are not flipped, this is same as cross correlation. In deep learning community "**convolution**" usually used to mean cross correlation.

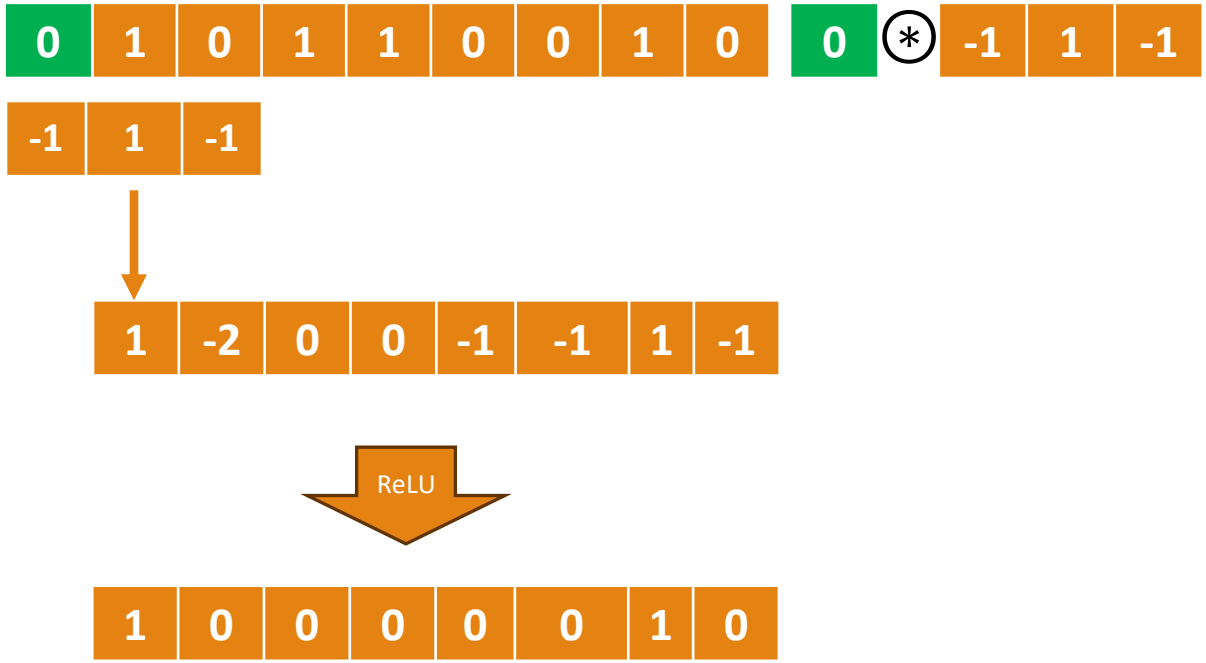
# Convolutional neural networks (CNNs)

## 1 D convolution



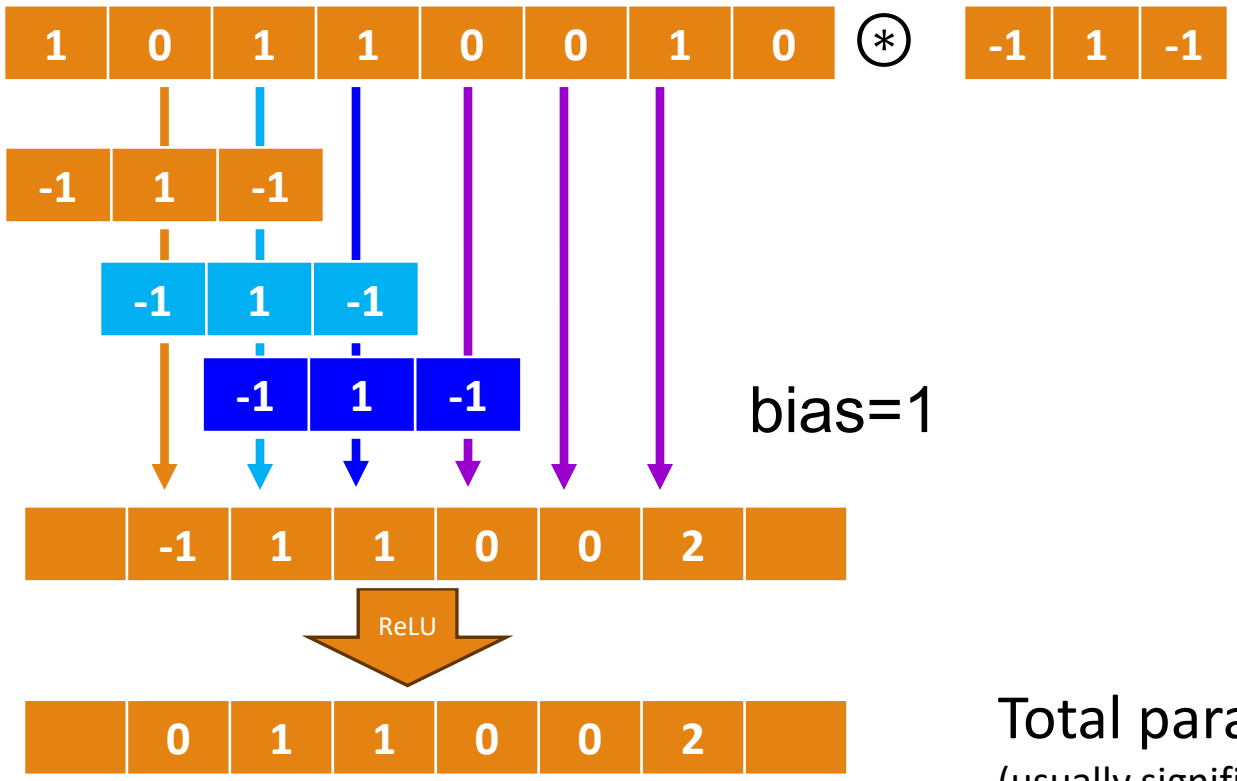

Detects isolated ones

## With padding

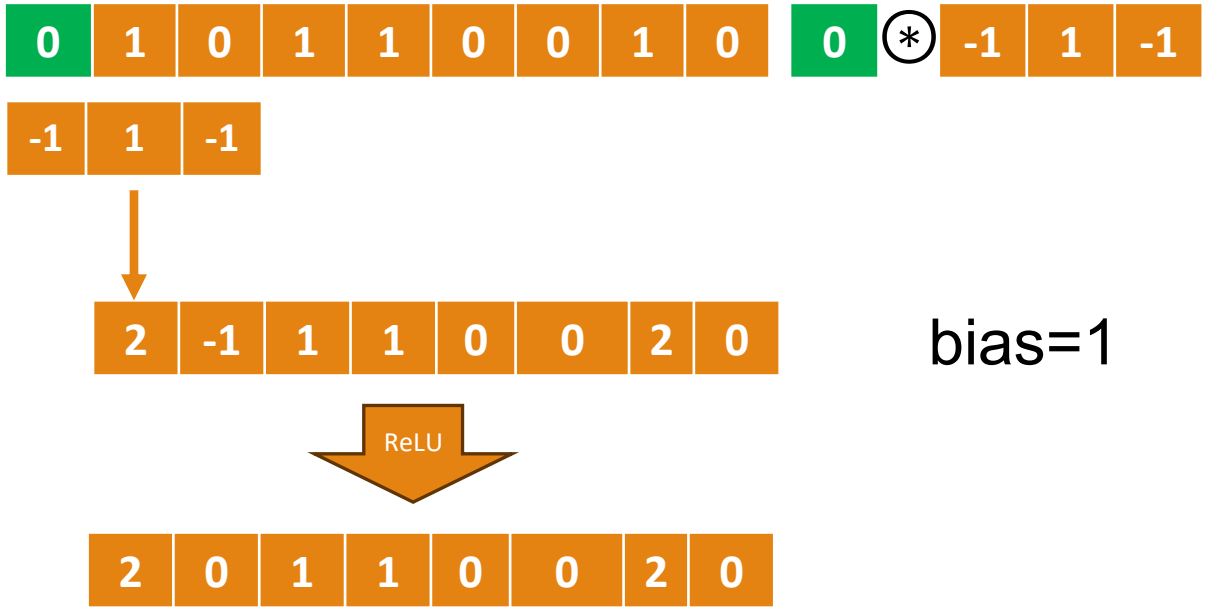


# Convolutional neural networks (CNNs)

1 D convolution with bias




With padding




Total parameters 4 ($w_1$, $w_2$, $w_3$, and $b_1$)

(usually significantly less than fully connected layer)



# Convolutional neural networks (CNNs)

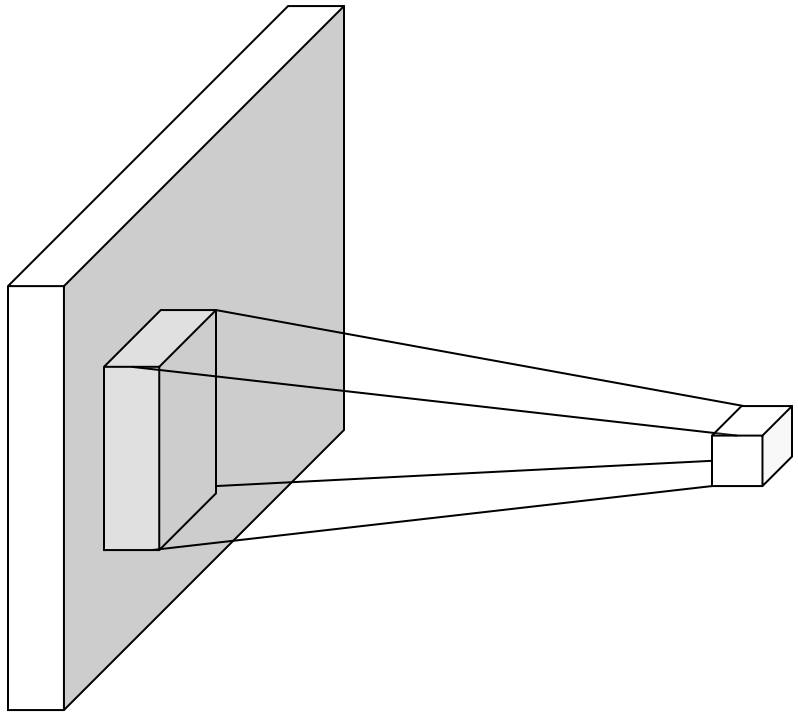
- 1 D convolution
  - Wiki Talk Comments Toxicity Prediction
- Audio Processing: Speech Recognition
- Time-Series Analysis
- Financial Data Analysis
- Biomedical Signal Processing: Genomic Sequence Analysis

Conv1D layer (keras.io)

tf.keras.layers.Conv1D | TensorFlow v2.14.0

Conv1d — PyTorch 2.1 documentation

# Convolutional architecture

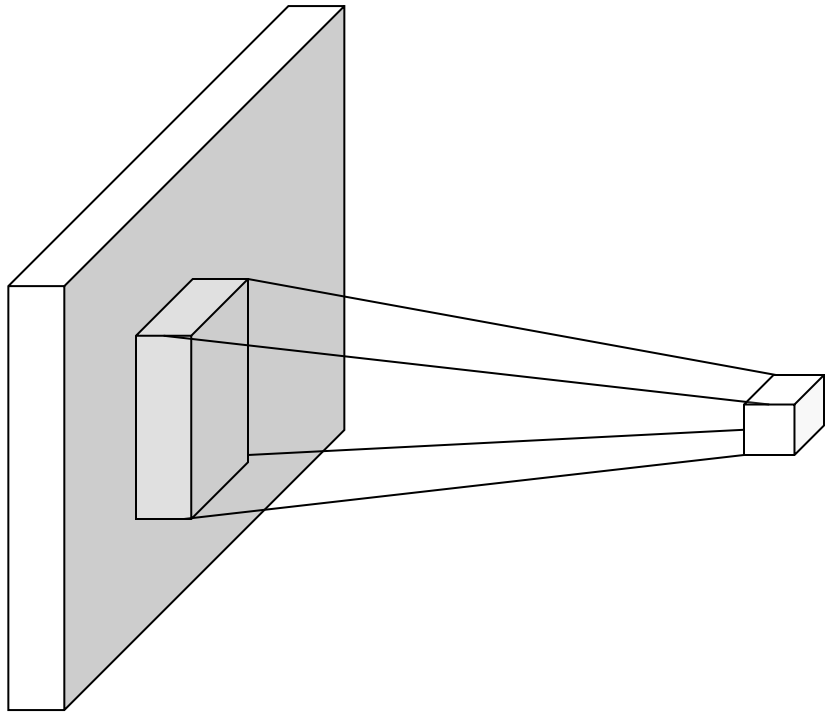


image

- Let's limit the *receptive fields* of units, tile them over the input image, and share their weights

Slide credit: Prof. Svetlana Lazebnik

# Convolutional architecture

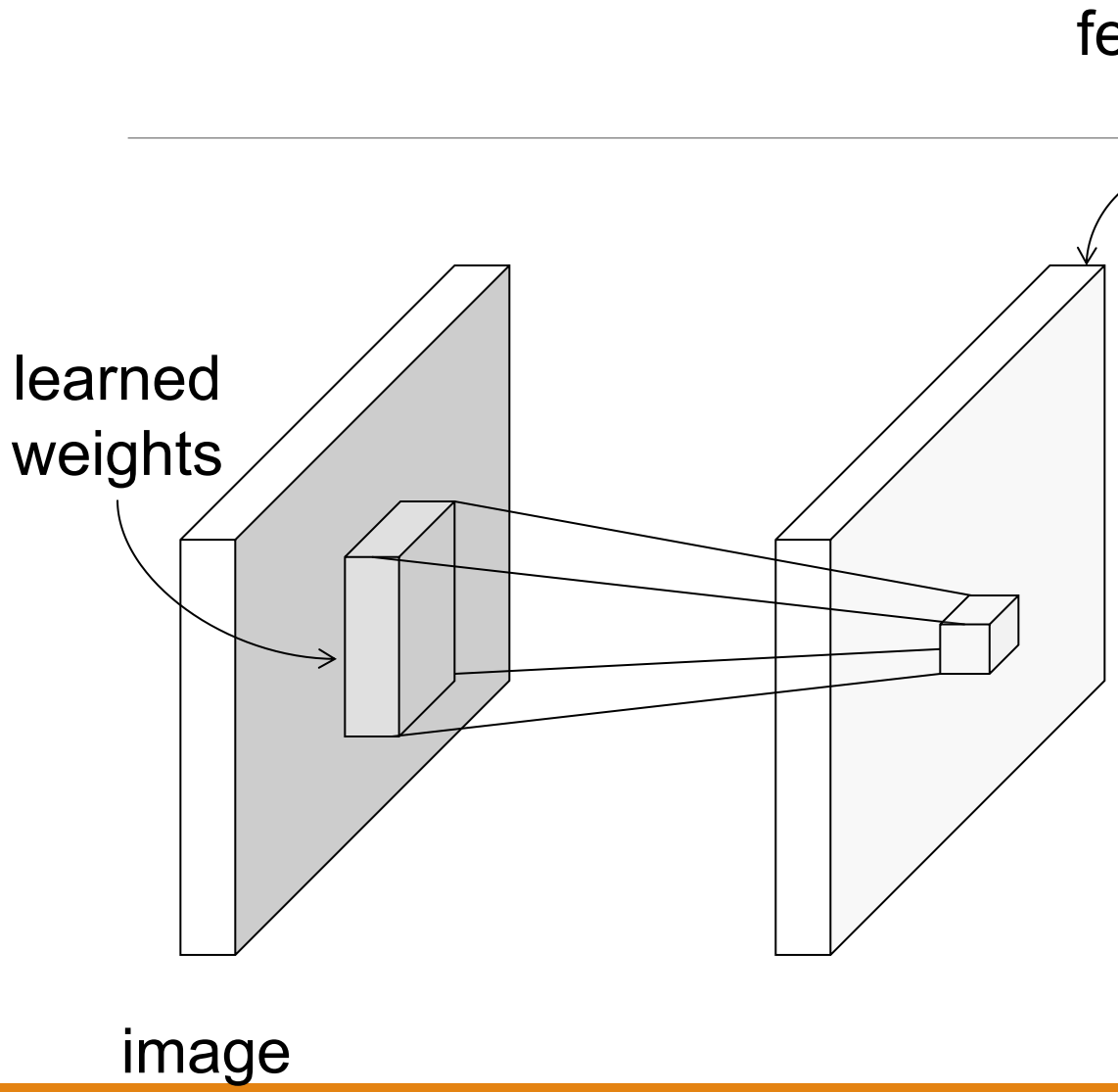


image

- Let's limit the *receptive fields* of units, tile them over the input image, and share their weights

Slide credit: Prof. Svetlana Lazebnik

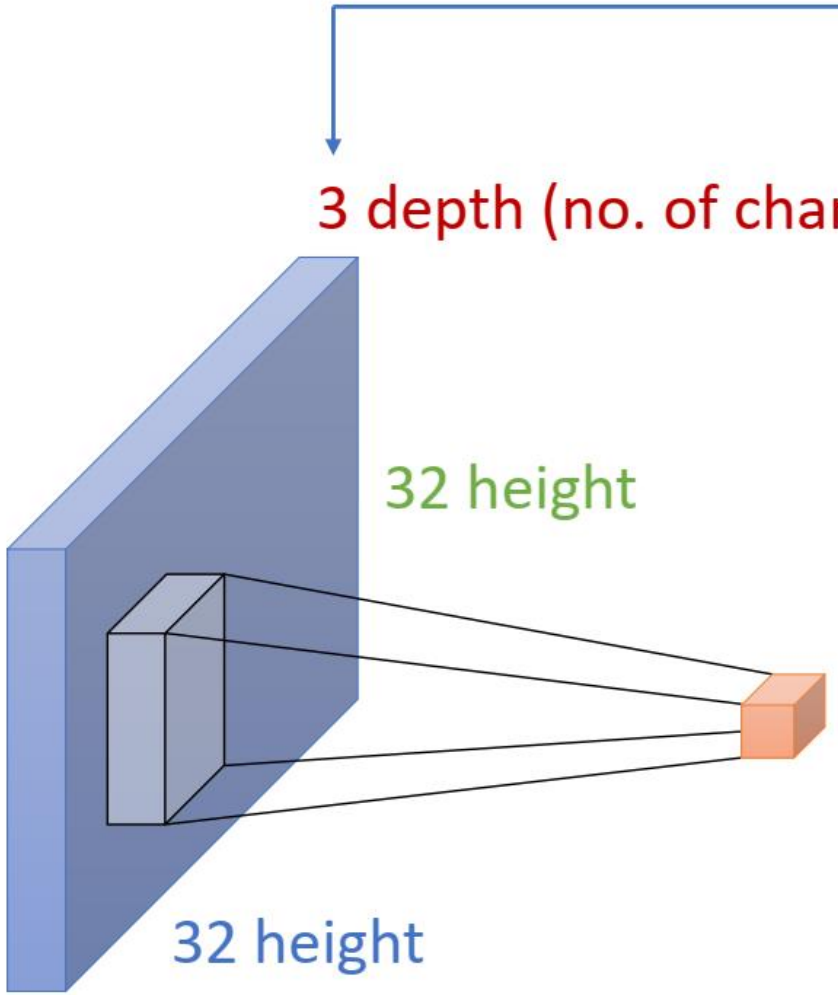
# Convolutional architecture




- Let's limit the *receptive fields* of units, tile them over the input image, and share their weights
- This is equivalent to sliding the learned filter over the image, computing dot products at every location

Slide credit: Prof. Svetlana Lazebnik

# Convolutional architecture




Extends the full depth of the image

3 × 5 × 5 kernel (filter)

$w^{\mathrm{T}}x + b$

$3 \times 5 \times 5 = 75$ dimensional dot product and addition of bias

Image

Convolutional layer

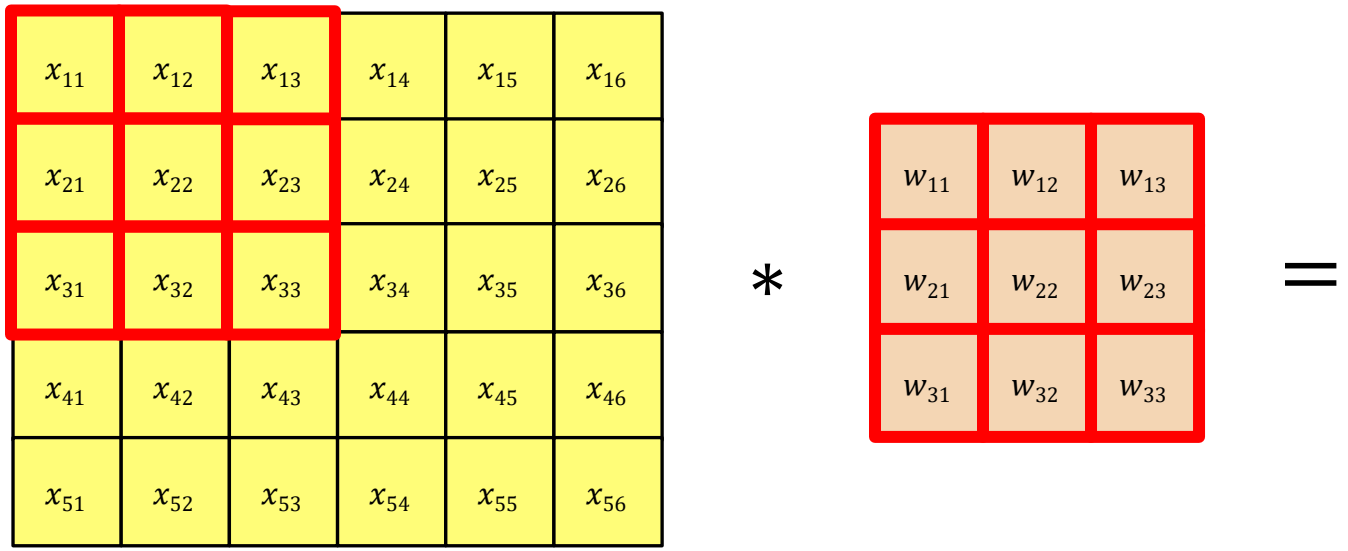
Slide credit: Dr. Justin Johnson

# Convolution example

Input

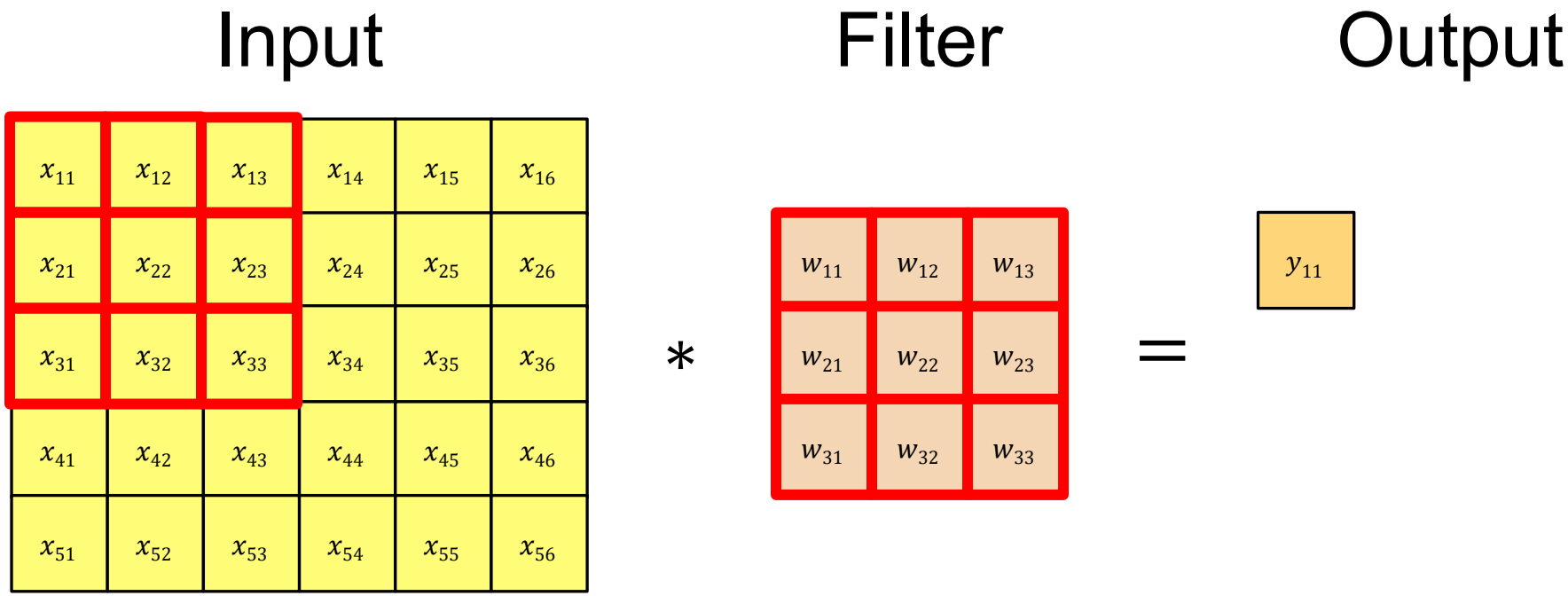
Filter

Output




Slide credit: Prof. Svetlana Lazebnik

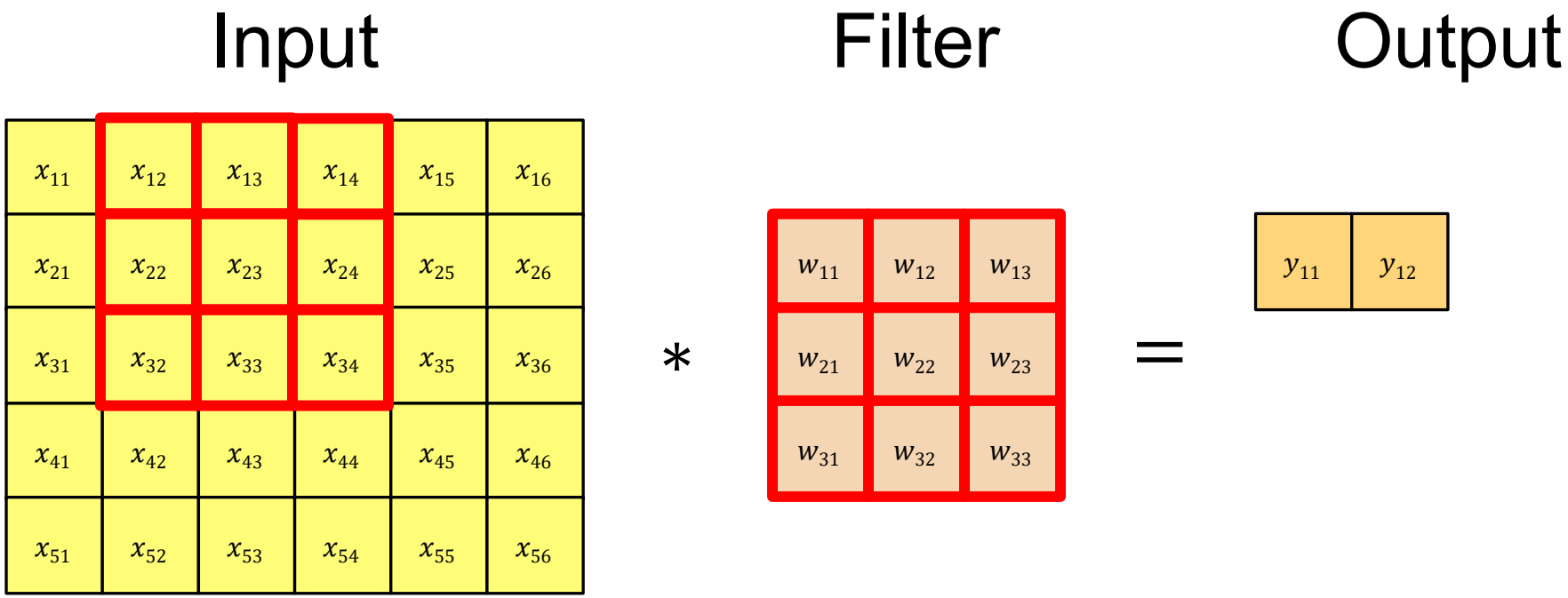
# Convolution example




$$y_{11} = x_{11} \cdot w_{11} + x_{12} \cdot w_{12} + x_{13} \cdot w_{13} + \ \ldots + x_{33} \cdot w_{33}$$

Slide credit: Prof. Svetlana Lazebnik

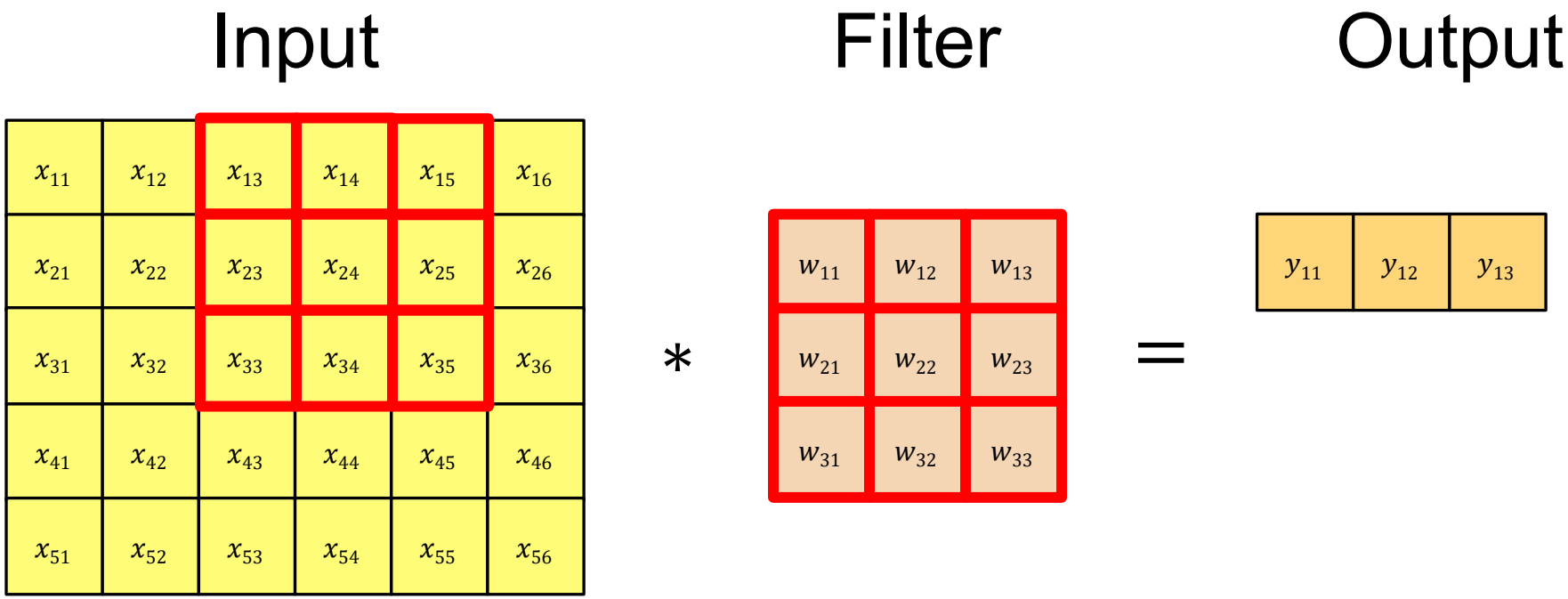
# Convolution example




$$y_{12} = x_{12} \cdot w_{11} + x_{13} \cdot w_{12} + x_{14} \cdot w_{13} + \ \ldots + x_{34} \cdot w_{33}$$

Slide credit: Prof. Svetlana Lazebnik

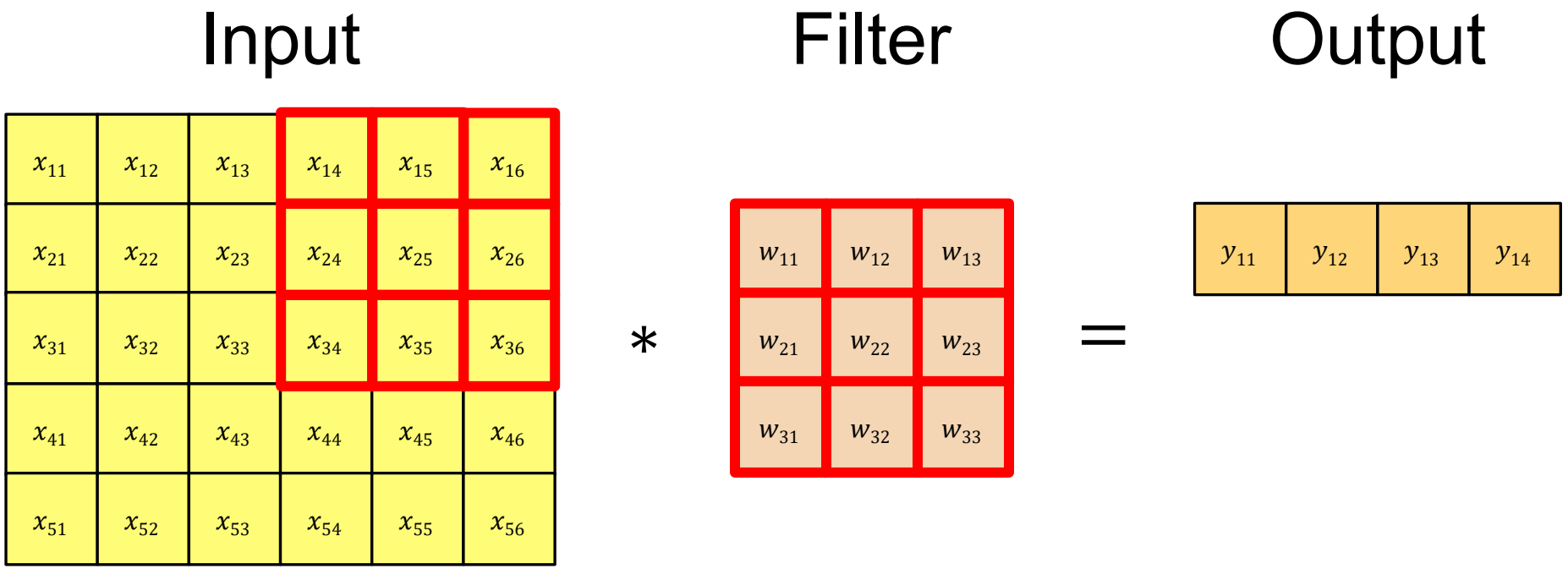
# Convolution example




$$y_{13} = x_{13} \cdot w_{11} + x_{14} \cdot w_{12} + x_{15} \cdot w_{13} + \ \ldots + x_{35} \cdot w_{33}$$

Slide credit: Prof. Svetlana Lazebnik

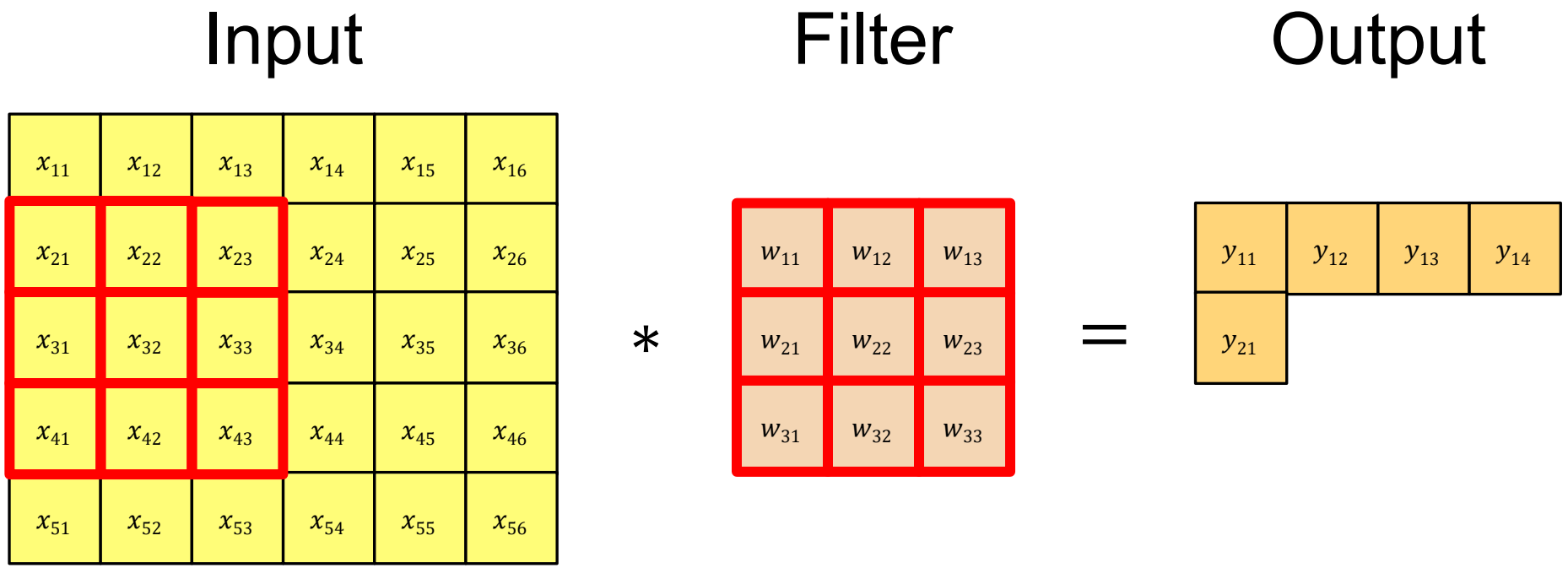
# Convolution example




$$y_{14} = x_{14} \cdot w_{11} + x_{15} \cdot w_{12} + x_{16} \cdot w_{13} + \ \ldots + x_{36} \cdot w_{33}$$

Slide credit: Prof. Svetlana Lazebnik

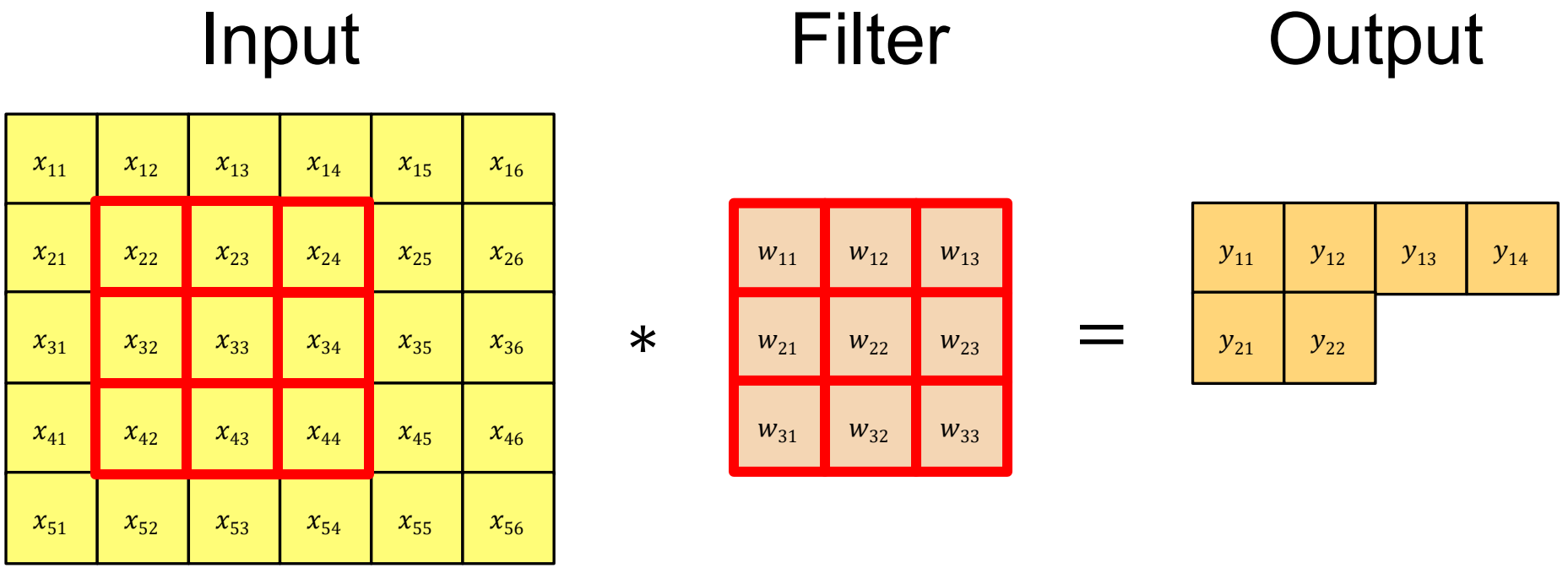
# Convolution example




$$y_{21} = x_{21} \cdot w_{11} + x_{22} \cdot w_{12} + x_{23} \cdot w_{13} + \ \ldots + x_{43} \cdot w_{33}$$

Slide credit: Prof. Svetlana Lazebnik

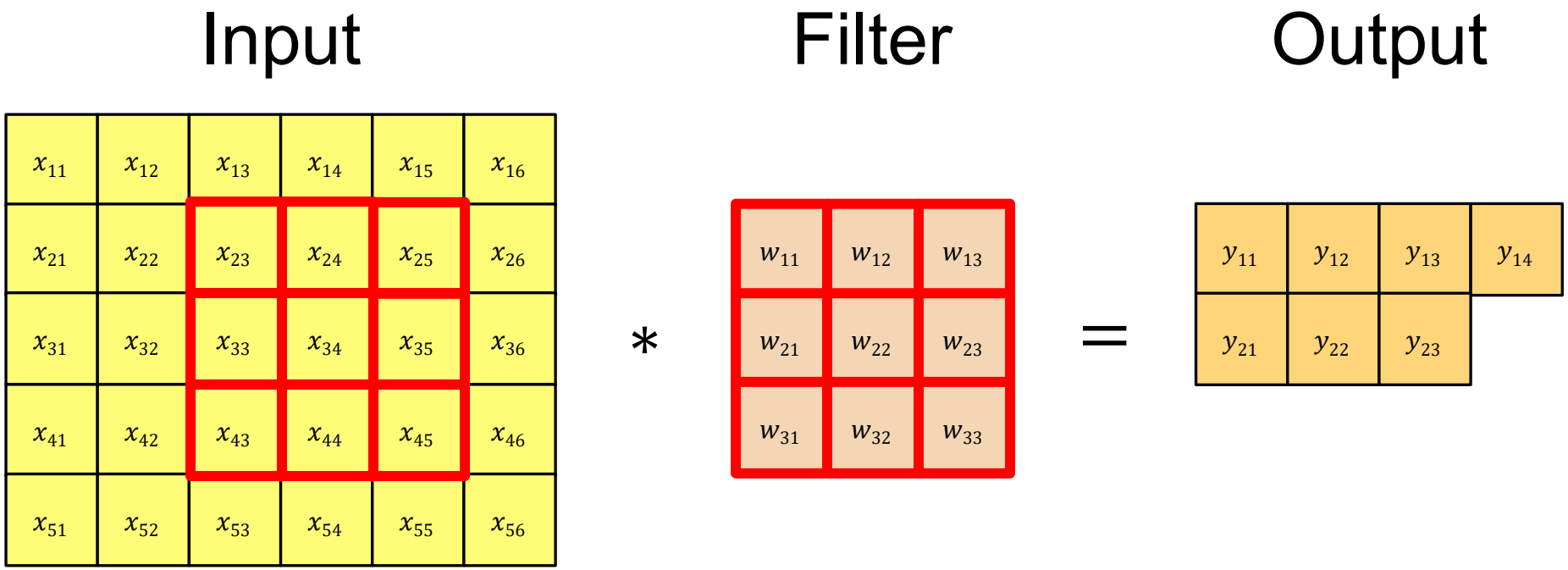
# Convolution example




$$y_{22} = x_{22} \cdot w_{11} + x_{23} \cdot w_{12} + x_{24} \cdot w_{13} + \ \ldots + x_{44} \cdot w_{33}$$

Slide credit: Prof. Svetlana Lazebnik

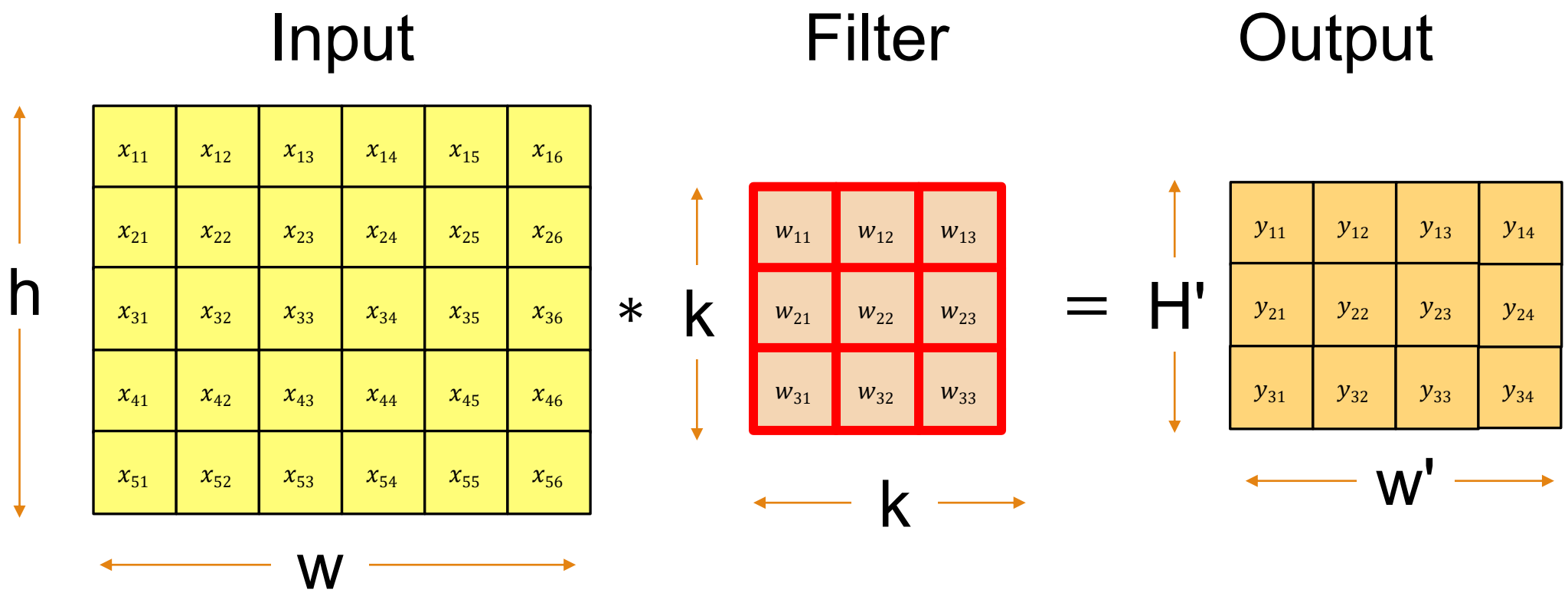
# Convolution example




$$y_{23} = x_{23} \cdot w_{11} + x_{24} \cdot w_{12} + x_{25} \cdot w_{13} + \ \ldots + x_{45} \cdot w_{33}$$

Slide credit: Prof. Svetlana Lazebnik

# Convolution example

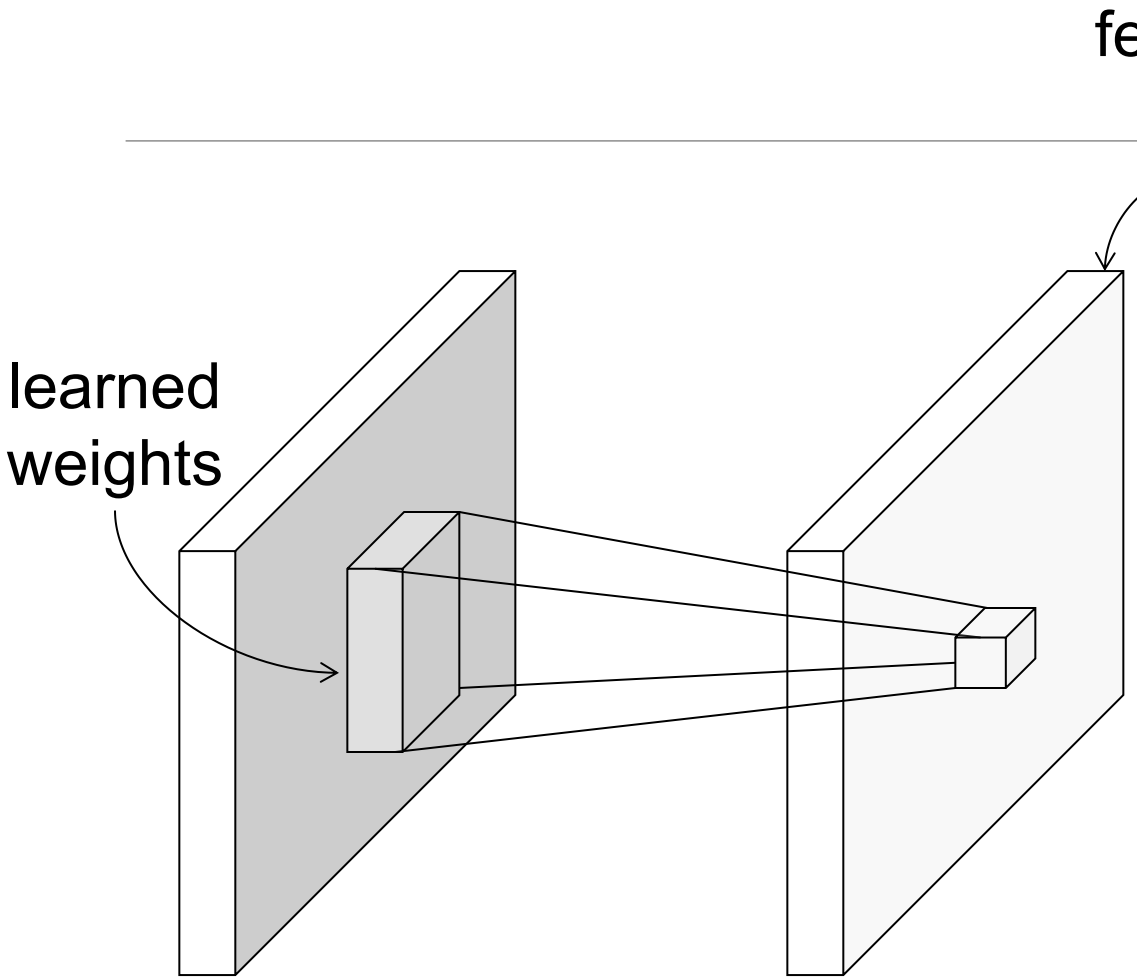



Number of multiplications = k x k x H' x w'

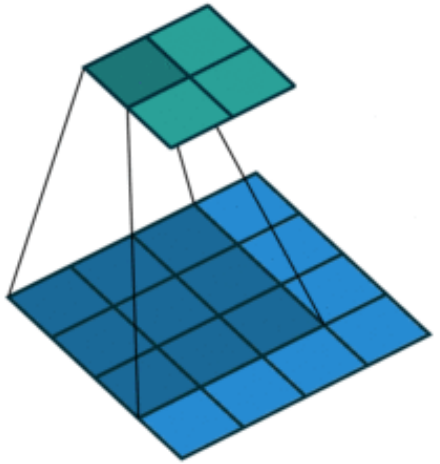
Number of additions (only for bias) = H' x w'

Slide credit: Prof. Svetlana Lazebnik

# Convolutional architecture




Output feature map resolution depends on *padding* and *stride*

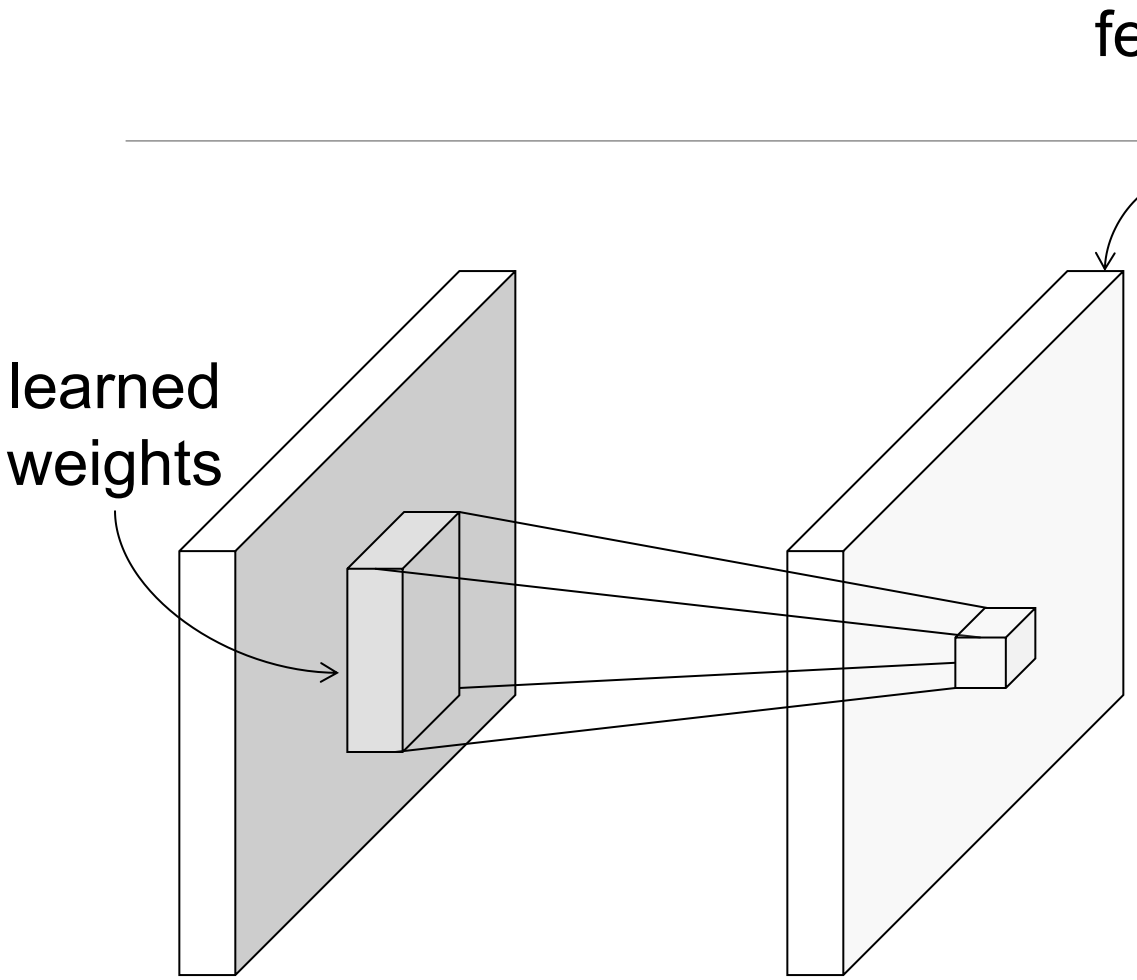



No padding, stride 1

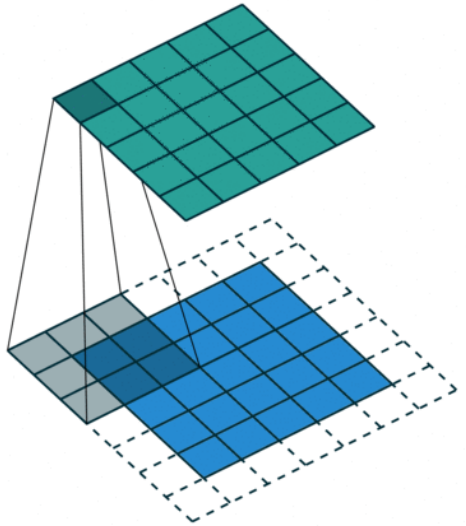
Slide credit: Prof. Svetlana Lazebnik

Animation source

# Convolutional architecture




Output feature map resolution depends on *padding* and *stride*

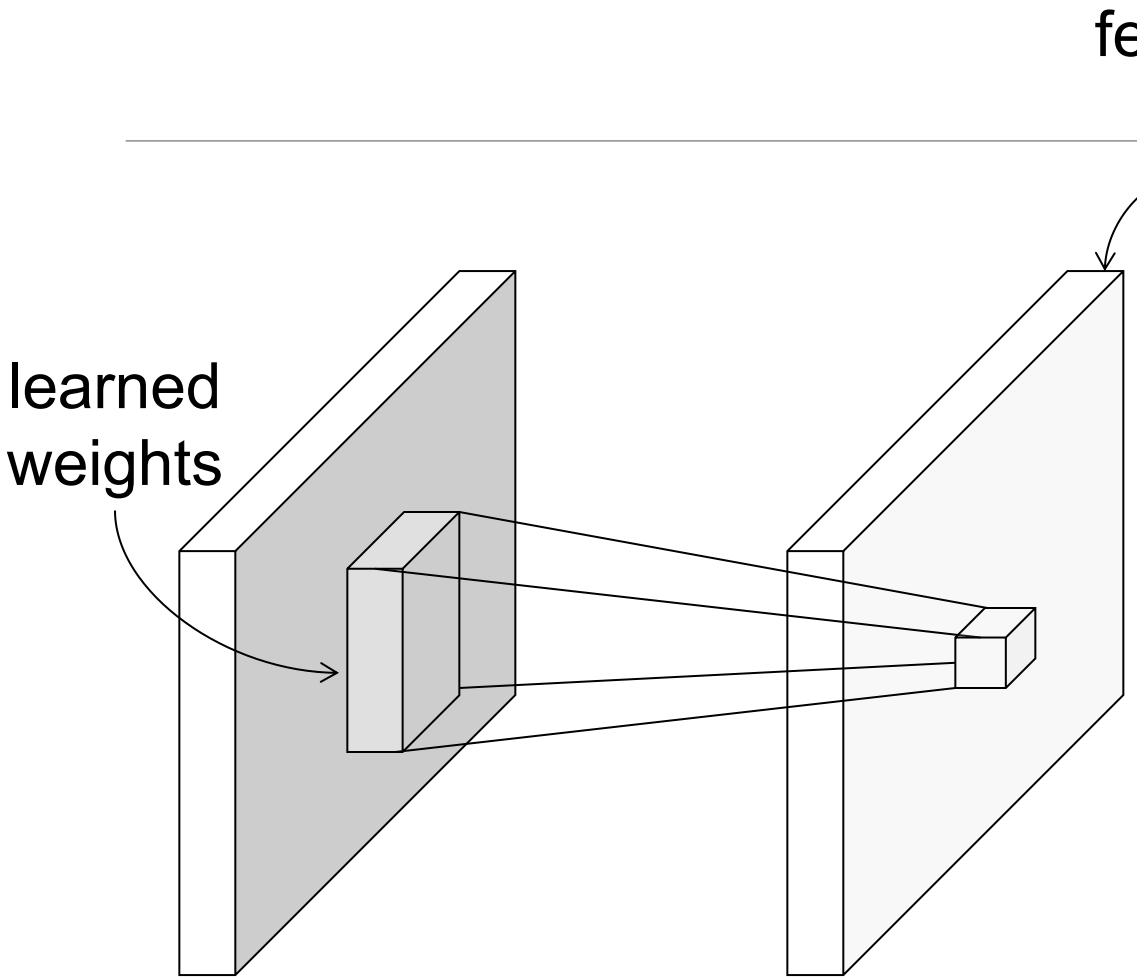



With padding, stride 1

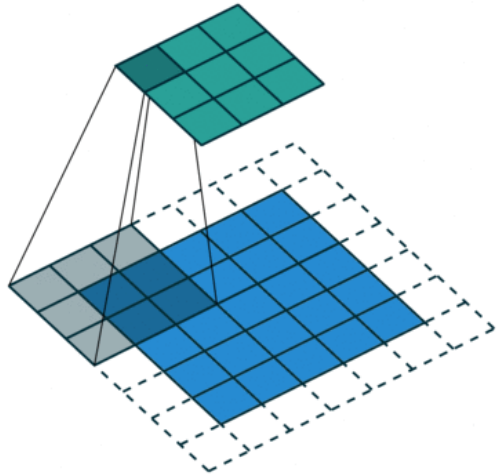
Slide credit: Prof. Svetlana Lazebnik

Animation source

# Convolutional architecture



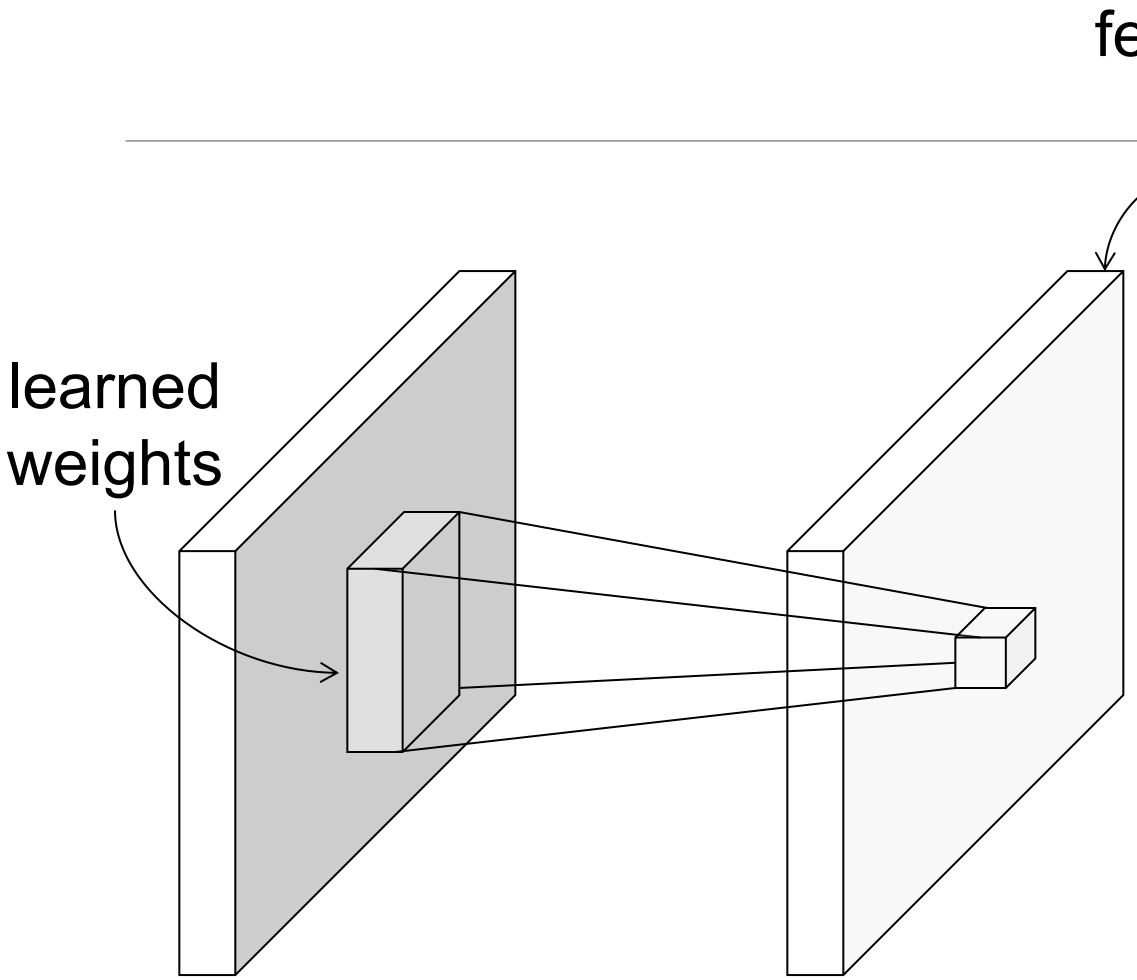

Output feature map resolution depends on *padding* and *stride*




With padding, stride 2

Slide credit: Prof. Svetlana Lazebnik

# Convolutional architecture



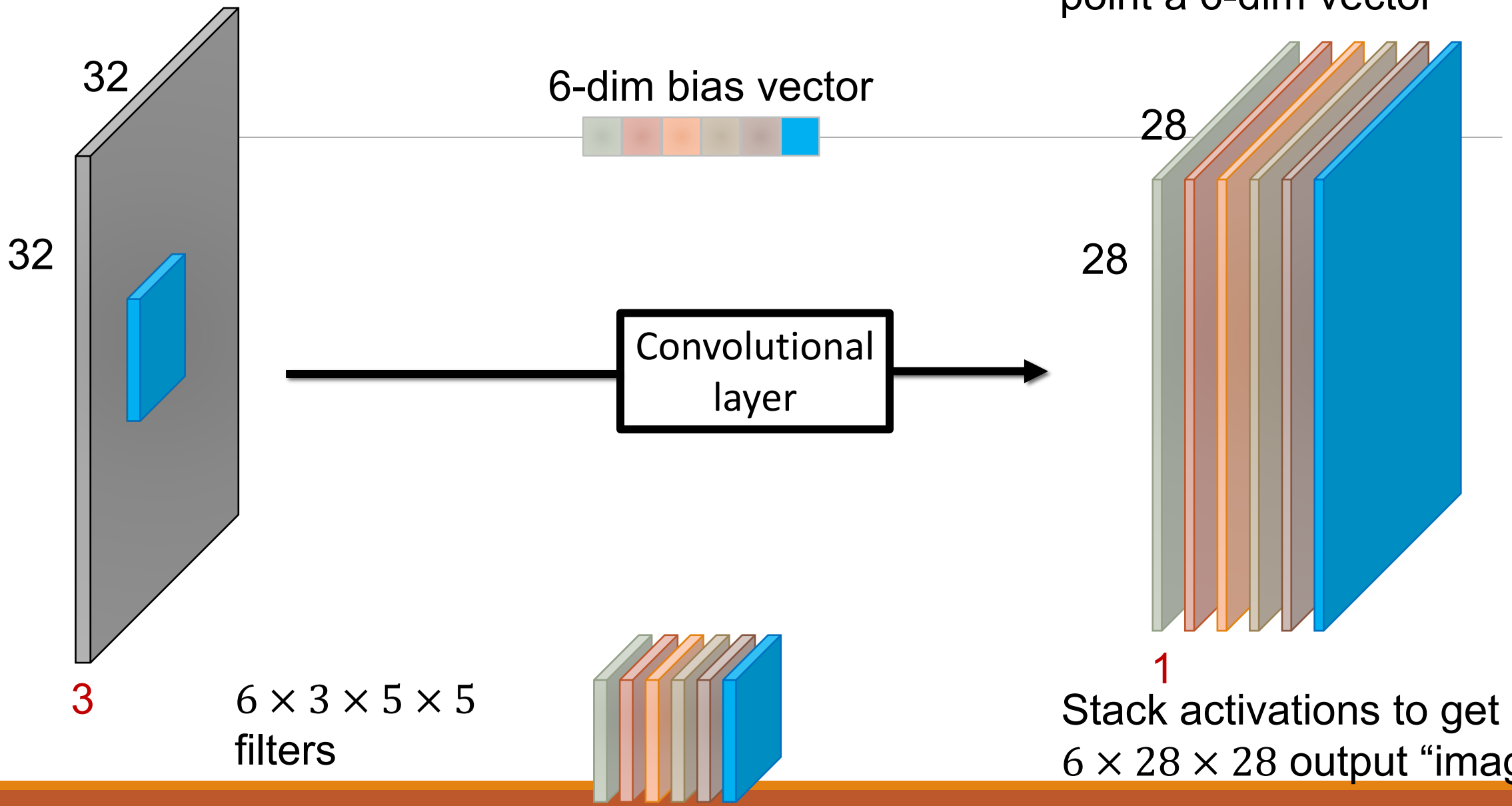

Output feature map resolution depends on *padding* and *stride*

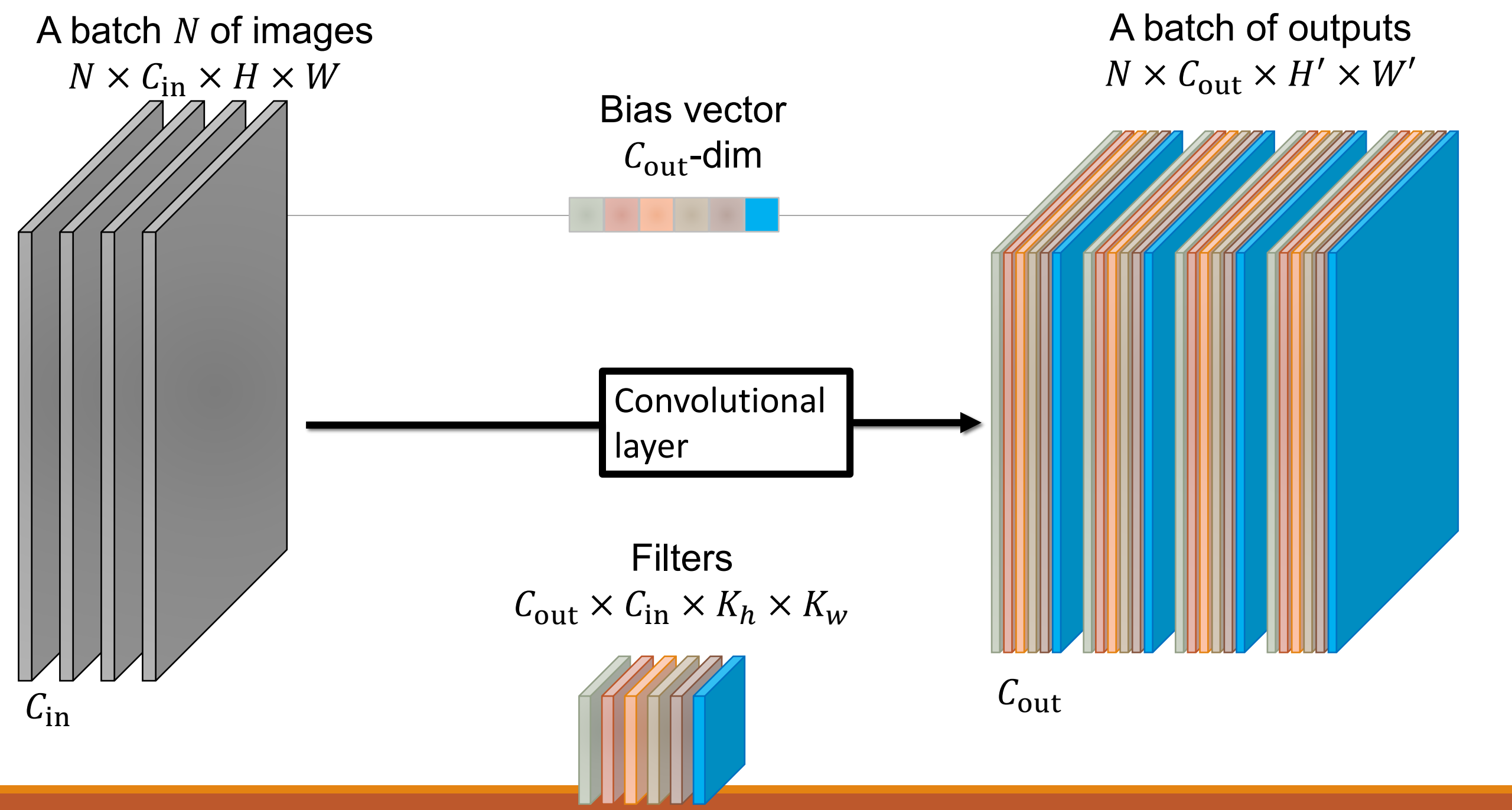
With padding, spatial resolution remains the same if stride of $1$ is used, is reduced by factor of $1/S$ if stride of $S$ is used

http://cs231n.github.io/convolutional-networks/#conv

Slide credit: Prof. Svetlana Lazebnik

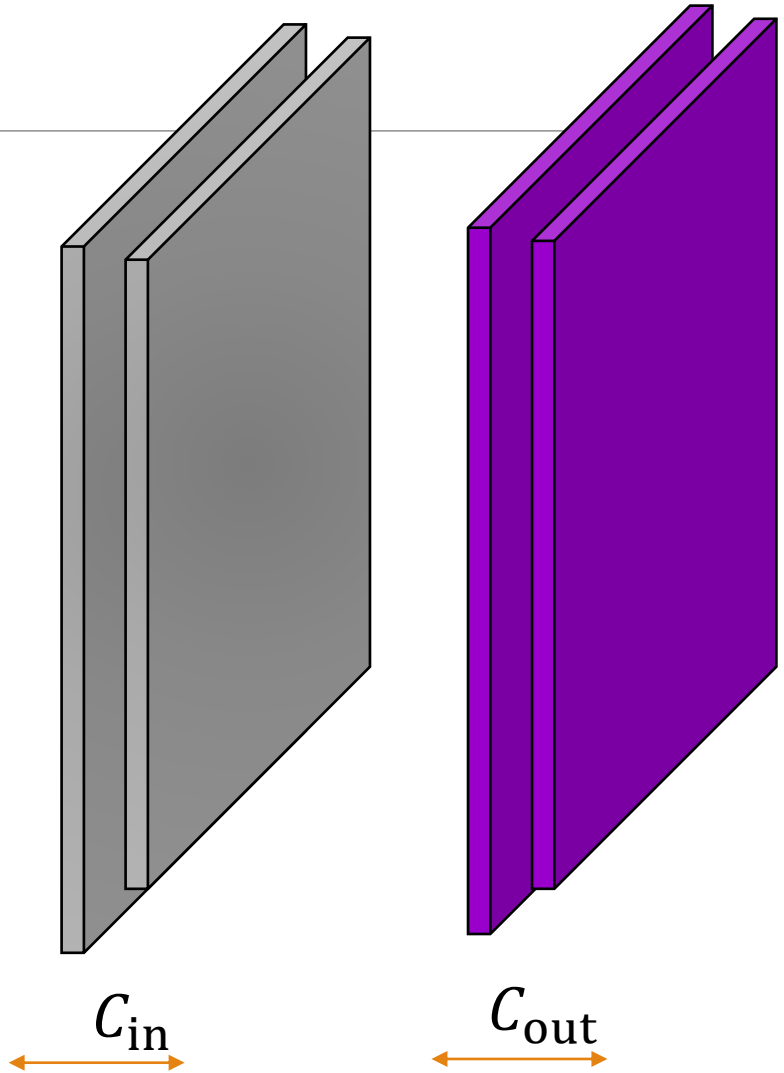
# Convolutional Layer




A batch $N$ of images
$N \times C_{\text{in}} \times H \times W$
Bias vector
$C_{\text{out}}$-dim
A batch of outputs
$N \times C_{\text{out}} \times H' \times W'$
Convolutional layer
Filters
$C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w$
$C_{\text{in}}$
$C_{\text{out}}$

# Convolution example

- With padding (same input and output size (HxW))
- For one feature map (input and output)
  Number of multiplications = $H \times W \times K_h \times K_w$
- Input feature maps $C_{\text{in}}$ and output feature maps $C_{\text{out}}$
- Number of multiplications = $C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w$ x $H \times W$
- $C_{\text{out}} = C_{\text{in}} = C$ and $K_h = K_w = K$
- Number of multiplications = $C^2$ $K^2$HW



# Learnt Filters

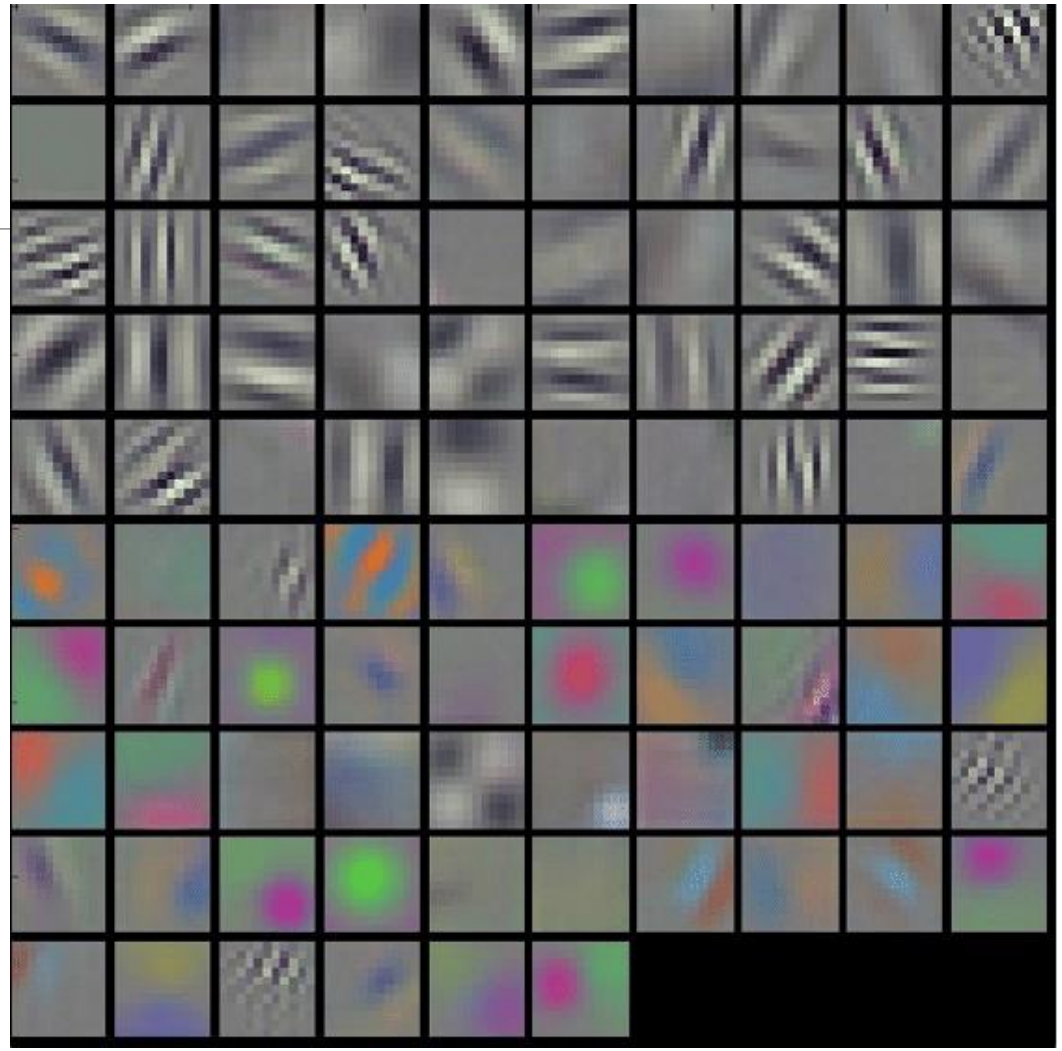



Linear classifier

https://cs231n.github.io/linear-classify/



Fully – connected
From Justin Johnson slides



AlexNet layer 1
11 × 11 weights

https://www.researchgate.net/figure/First-layer-filters-of-AlexNet-trained-on-ImageNet-A-considerable-amount-of-redundancy_fig4_338162455

# Convolution as Feature Extraction



Input

Feature Map

Slide: Prof. Svetlana Lazebnik

# Visualizing CNNs

- Demo: CNN Explainer (poloclub.github.io)
- Demo Video "CNN Explainer: Learning Convolutional Neural Networks with Interactive Visualization"
- Text: CNN Explainer




- CS231n Convolutional Neural Networks for Visual Recognition
- Visualizing and Understanding Convolutional Networks

# Known Kernels

```
[1/9, 1/9, 1/9],
[1/9, 1/9, 1/9],
[1/9, 1/9, 1/9]
```

```
[-1, -2, -1],
[ 0,  0,  0],
[ 1,  2,  1]
```

```
[-1, 0, 1],
[-2, 0, 2],
[-1, 0, 1]
```





Box blur kernel



Sobel y-kernel



Sobel x-kernel



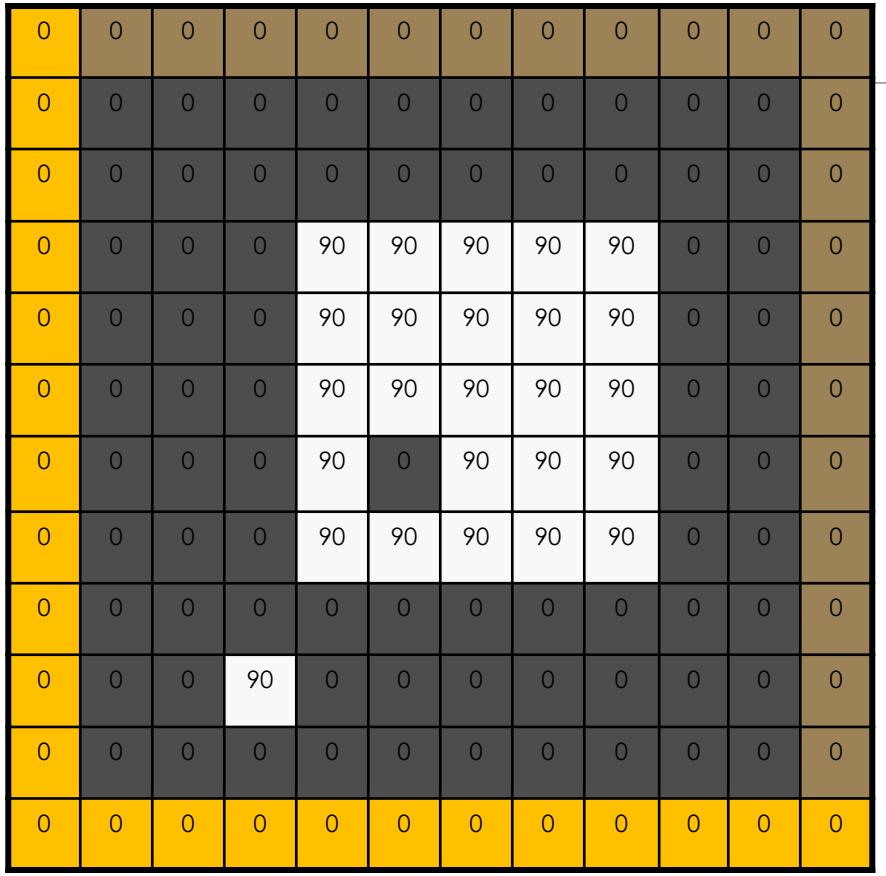
Gaussian blur kernel



Emboss kernel

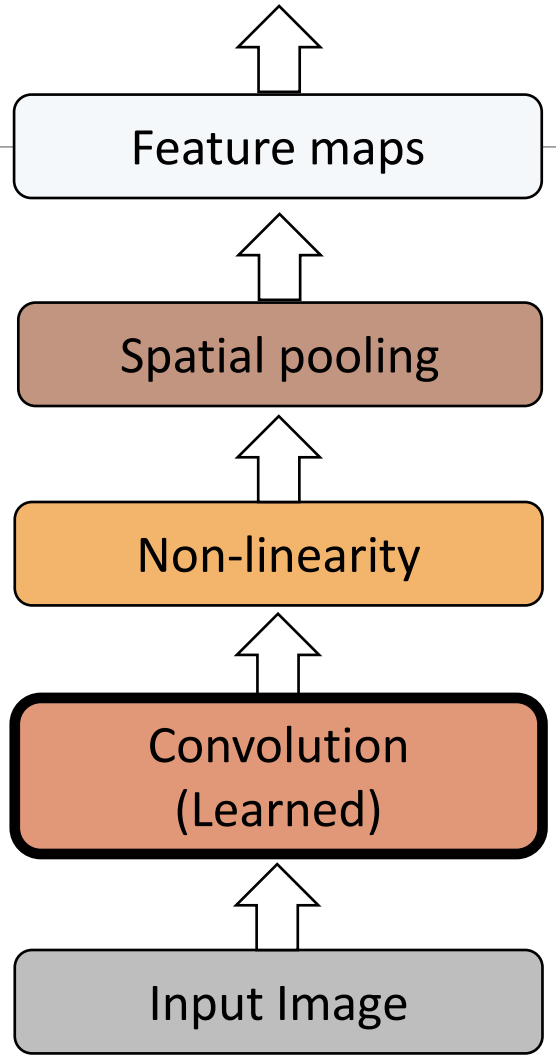
https://colab.research.google.com/drive/1pHxYZ7VOObPJx_zrMk_vg4fYNUKvTa_i?usp=sharing

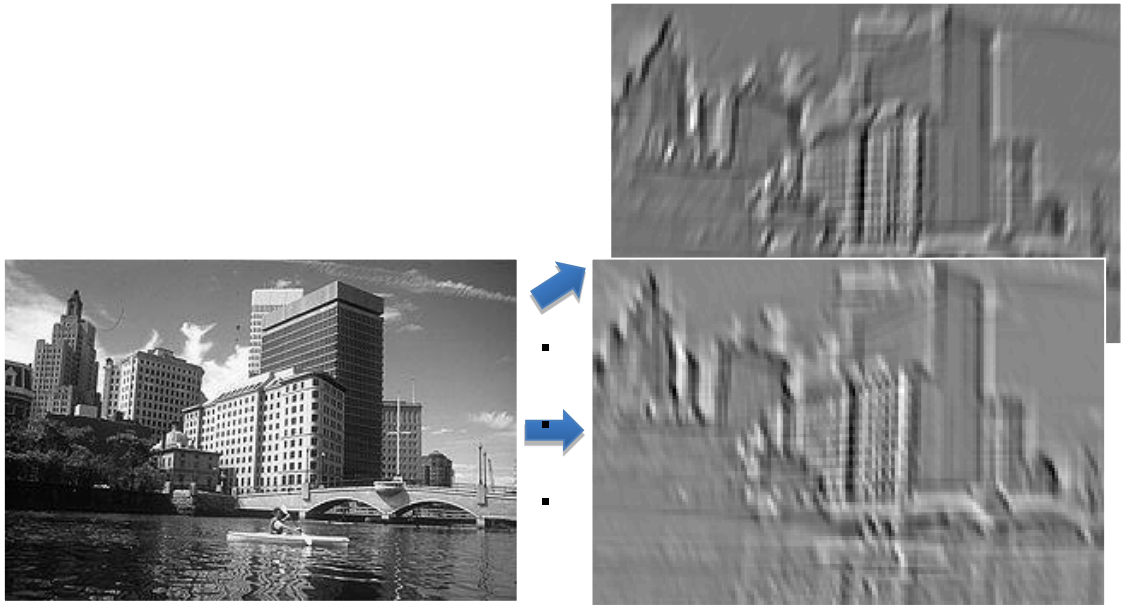
# Shrinking Due to Convolutions and Padding




Input: 10x10
Filter: 3x3
Output: 8x8
In general:
Input: $W$ ($W \times W$)
Filter: $K$ ($K \times K$)
Output: W – K + 1
Padding: $P$
Output: $W - K + 1 + 2P$
Set $P = (K - 1) / 2$ to
make output have same size as input.

# Key Operations in a CNN

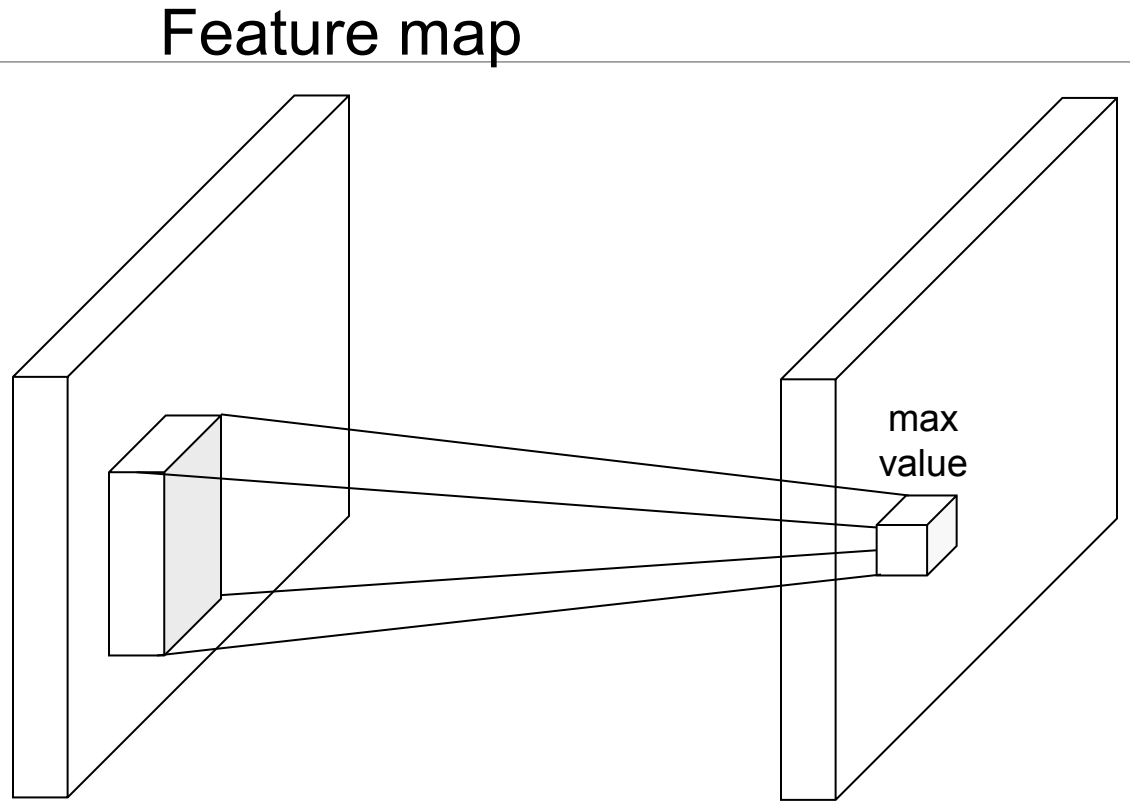



Source: R. Fergus, Y. LeCun

Slide: Lazebnik

# Max pooling layer




$F \times F$ pooling window, stride $S$

Usually: $F = 2$ or $3$, $S = 2$

Slide credit: Prof. Svetlana Lazebnik

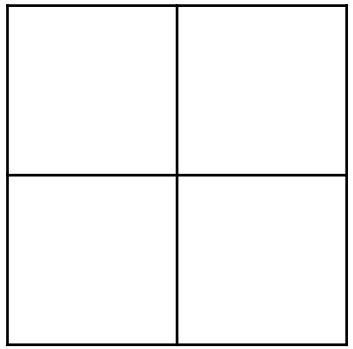
# Max pooling: Example

Single channel

| | | | |
|---|---|---|---|
| 1 | 1 | 2 | 4 |
| 5 | 6 | 7 | 8 |
| 3 | 2 | 1 | 0 |
| 1 | 2 | 3 | 4 |

Max pooling with $2 \times 2$ kernel size and stride 2





Slide credit: Prof. Svetlana Lazebnik

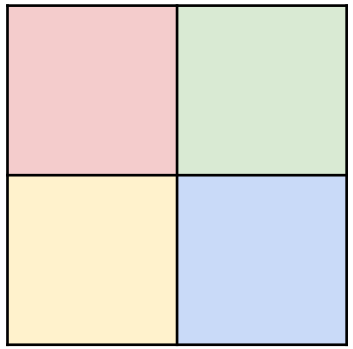
Source: J. Johnson

# Max pooling: Example

Single channel

| | | | |
|---|---|---|---|
| 1 | 1 | 2 | 4 |
| 5 | 6 | 7 | 8 |
| 3 | 2 | 1 | 0 |
| 1 | 2 | 3 | 4 |

Max pooling with $2 \times 2$ kernel size and stride 2



Slide credit: Prof. Svetlana Lazebnik

# Max pooling: Example

Single channel

| | | | |
|---|---|---|---|
| 1 | 1 | 2 | 4 |
| 5 | 6 | 7 | 8 |
| 3 | 2 | 1 | 0 |
| 1 | 2 | 3 | 4 |

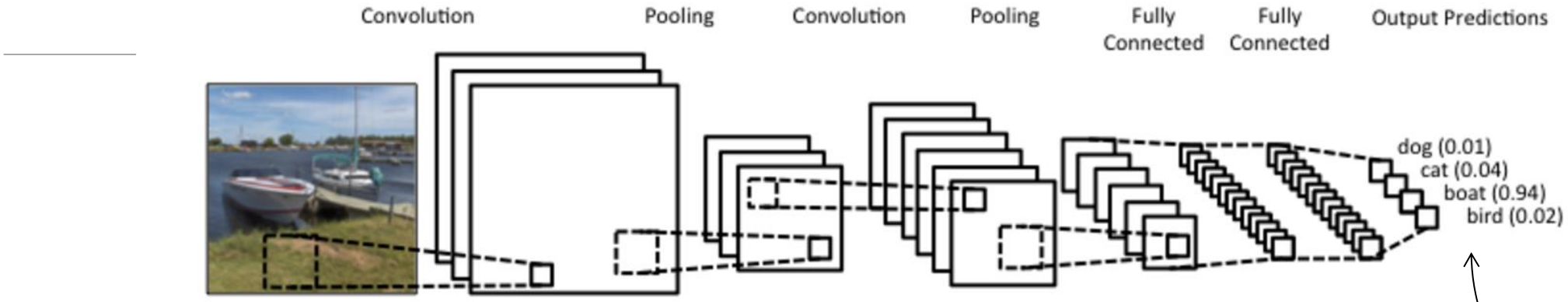
Max pooling with $2 \times 2$ kernel size and stride 2



| | |
|---|---|
| 6 | 8 |
| 3 | 4 |

Slide credit: Prof. Svetlana Lazebnik

# Simplified CNN pipeline




Softmax layer:

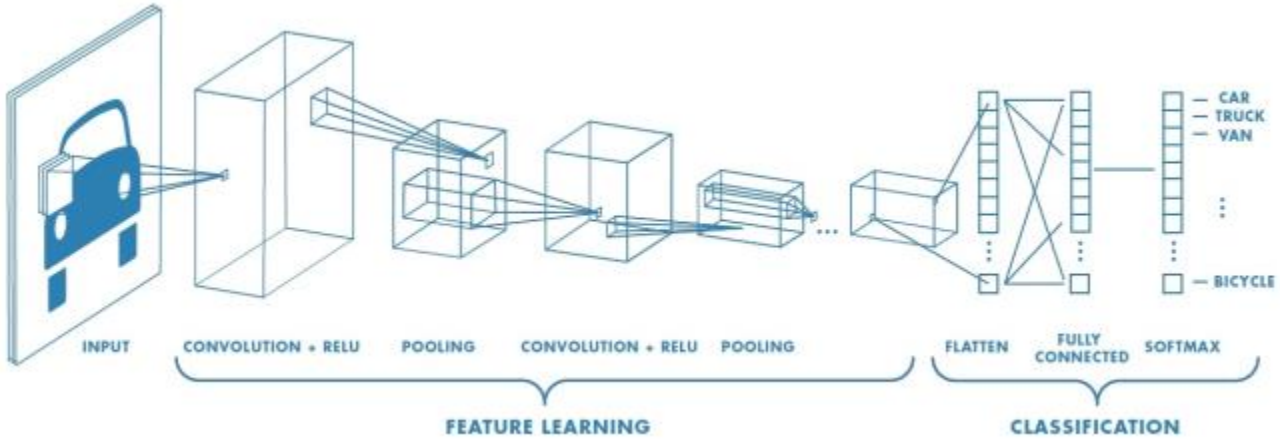


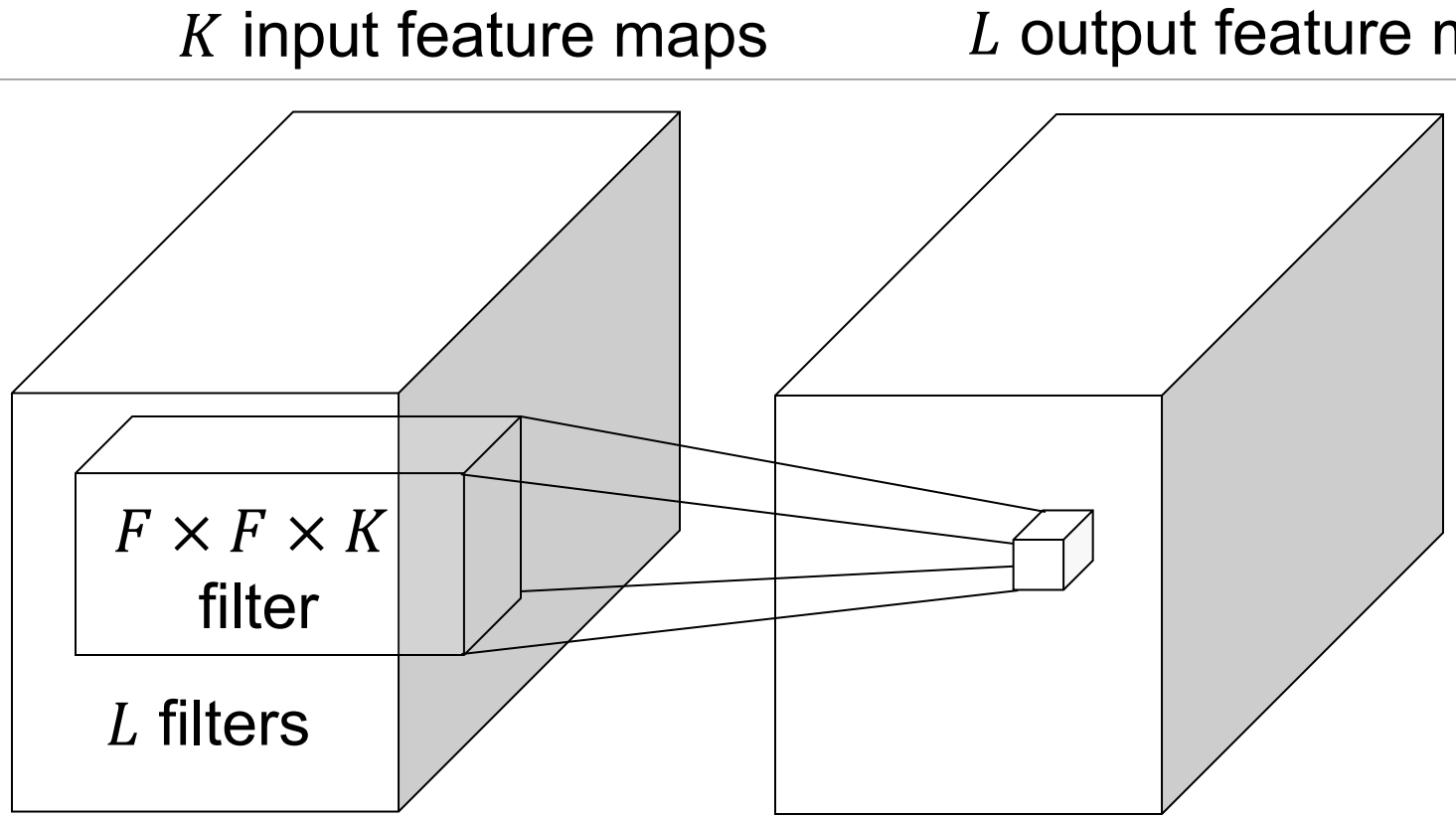

Image adapted from https://blog.floydhub.com

[What are Convolutional Neural Networks (CNNs)]

[MIT 6.S191: Convolutional Neural Networks]

Modified slide from Prof. Svetlana Lazebnik

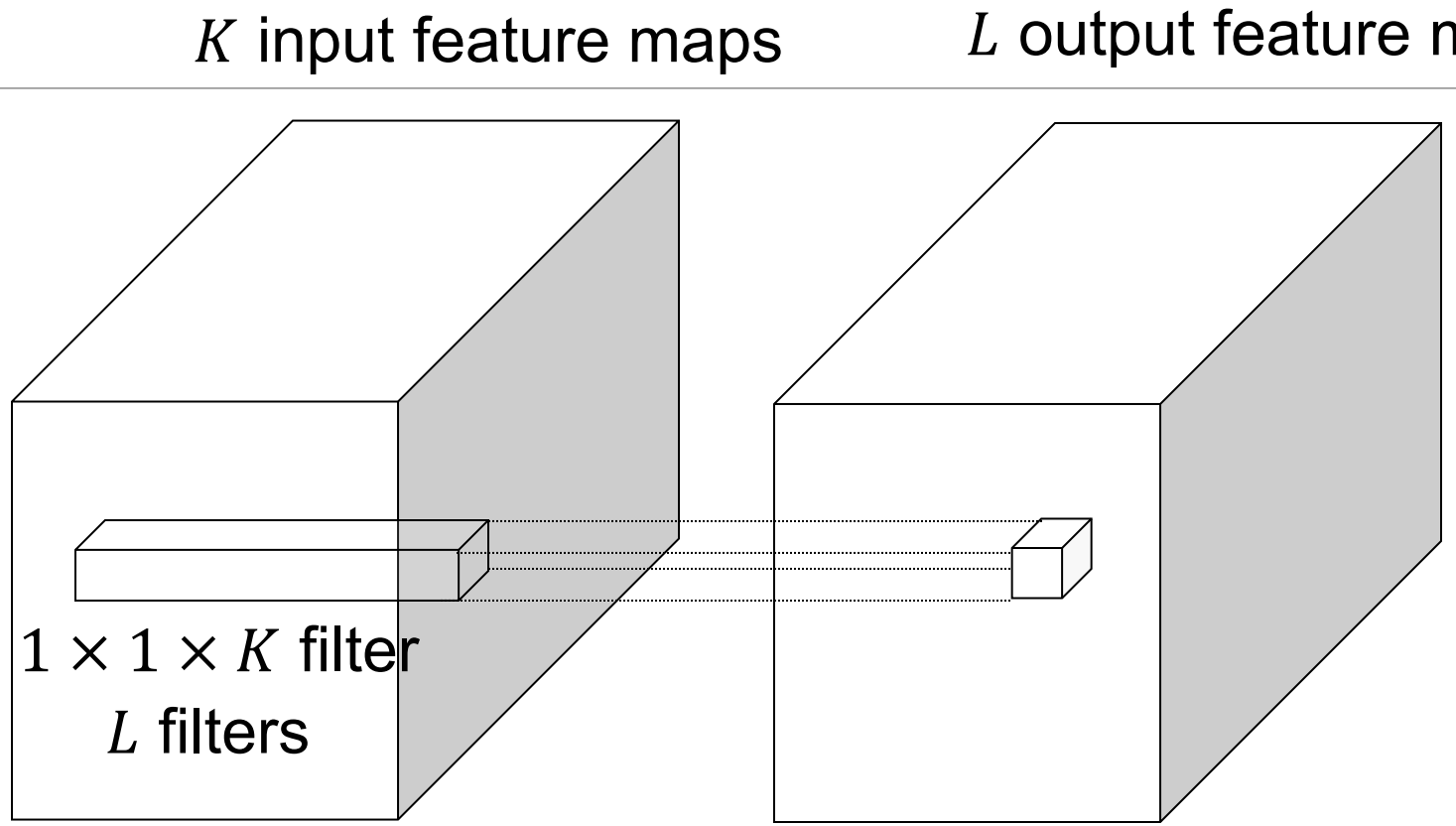
# 1x1 convolutional layer




What if we make $F = 1$?

Slide credit: Prof. Svetlana Lazebnik

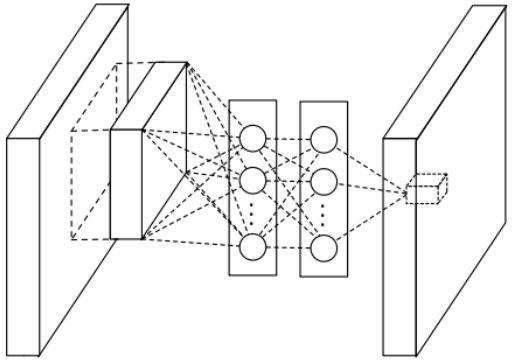
# 1x1 convolutional layer



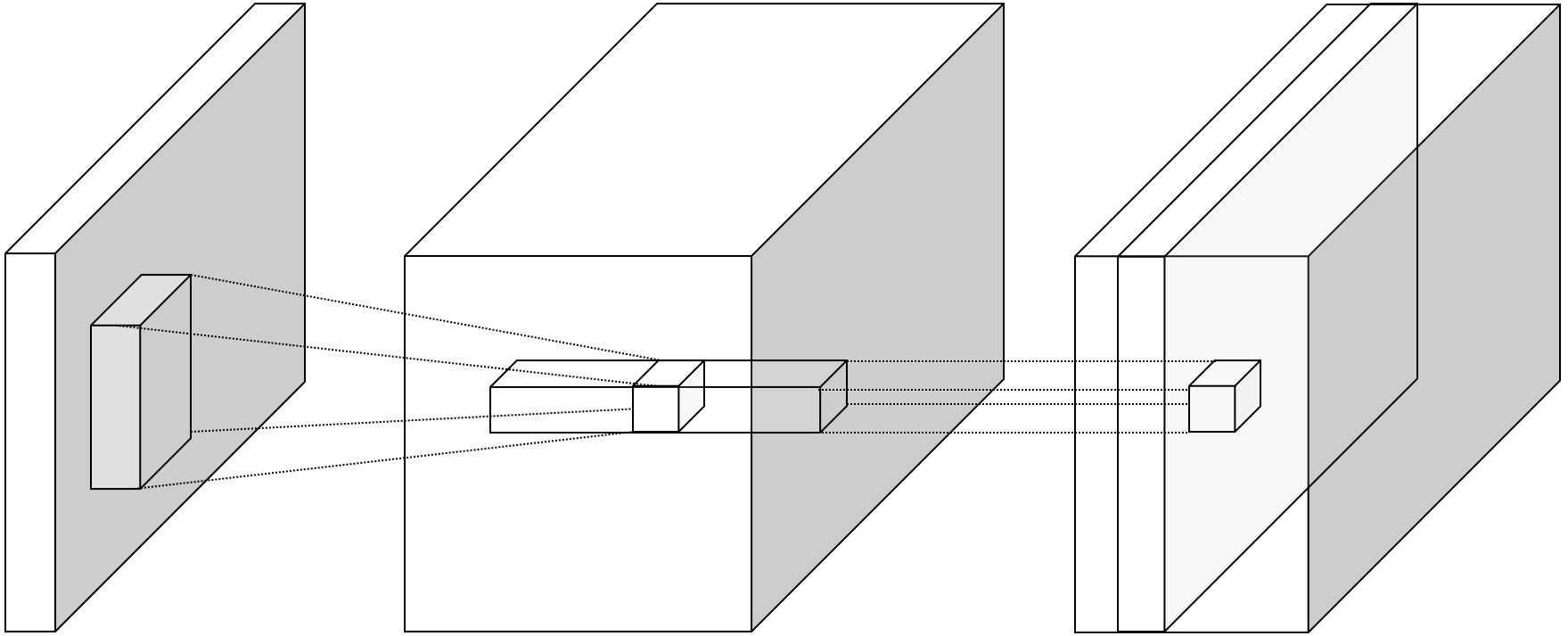

What if we make $F = 1$?

Slide credit: Prof. Svetlana Lazebnik

# 1x1 convolutional layer



"network in network"



Slide credit: Prof. Svetlana Lazebnik

M. Lin, Q. Chen, and S. Yan, Network in network, ICLR 2014

# Why 1x1 convolutions?

- **Option 1:** $3 \times 3$ conv layer with $256$ channels at input and output
  - $256 \times 256 \times 3 \times 3 \approx 600{,}000$ weights and operations (per location)

- **Option 2: "bottleneck module"**

  $1 \times 1$ conv layer, $256 \rightarrow 64$ channels

  $3 \times 3$ conv layer, $64 \rightarrow 64$ channels

  $1 \times 1$ conv layer, $64 \rightarrow 256$ channels

  - $256 \times 64 \times 1 \times 1 \approx 16{,}000$
  - $64 \times 64 \times 3 \times 3 \approx 36{,}000$
  - $64 \times 256 \times 1 \times 1 \approx 16{,}000$
  - Total $\approx 70{,}000$ weights and operations

**Dimension Reduction**

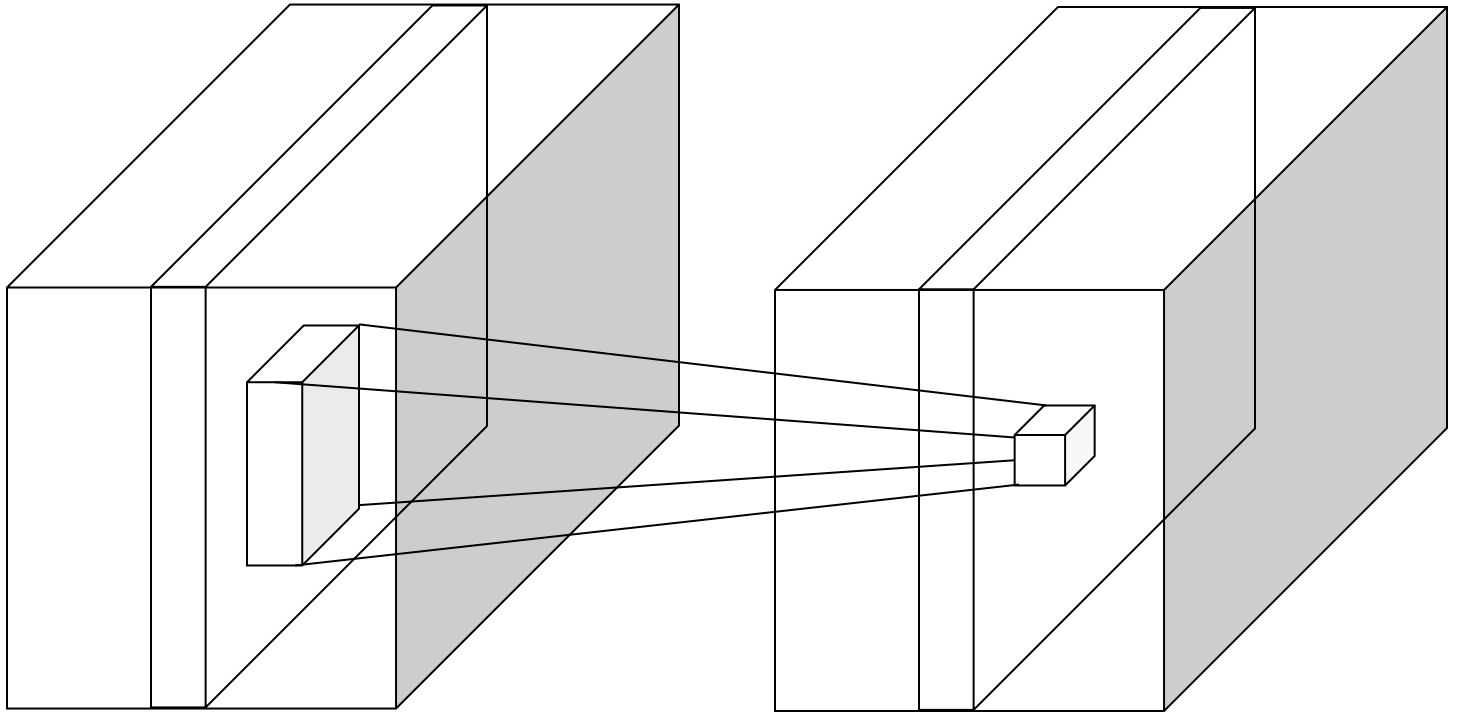
Slide credit: Prof. Svetlana Lazebnik

# Alternative to 3D convolutions: Depth-wise convolutions

$K$ input feature maps

$K$ output feature maps

$K$ $F \times F$ filters



- For an input with k channels, k different filters are used, each of size $F \times F$
- Pointwise Convolution

applies a 1×1 convolution across all channels to combine the outputs from the depth-wise step

$F^2KHW$ instead of $F^2K^2HW$ operations

Can be followed by a 1D convolution to mix values across channels

operate on individual channels of multi-channel input data

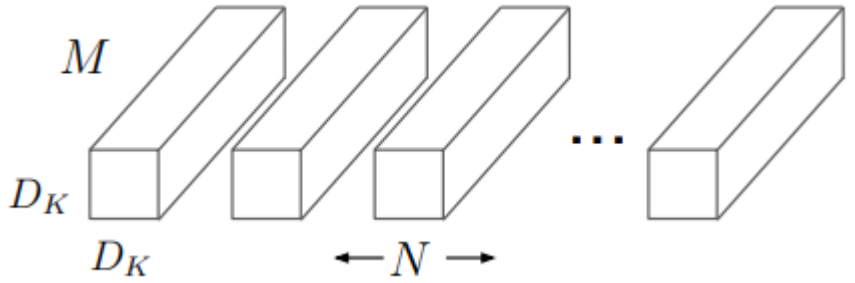
Slide credit: Prof. Svetlana Lazebnik

# MobileNets: Depth-wise separable convolutions

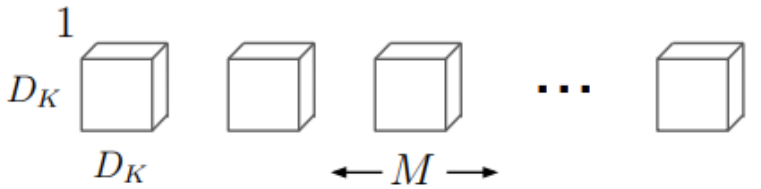
- **Depthwise separable convolution block:**
  3× 3 conv layer, 256 → 256 channels (for example)
  1 × 1 conv layer, $K \rightarrow K$ channels
  - **Step 1: 3× 3 conv layer of input channels**
    - **$9 \times 256 \approx 2,300$ operations (per location)**
  - **Step 2: 1× 1 conv layer of input channels x output channels**
    - $256 \times 256 \approx 65{,}500$ operations
  - Total $\approx \mathbf{2,300} + 65{,}500 = 67{,}800$ operations

- Regular 3 × 3 conv layer, 256 → 256 channels
  - $9 \times 256 \times 256 \approx 590{,}000$ operations

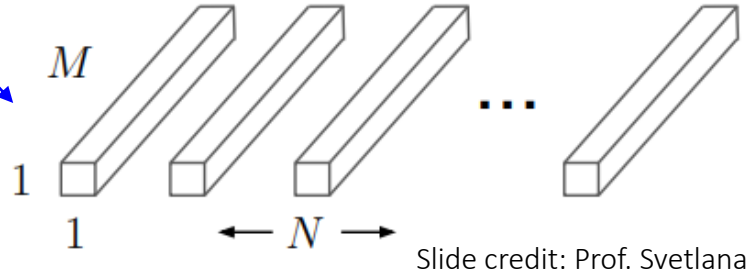
- Speedup factor: 8.7




Standard Convolution Filters,
M input channels and N output channels [1]



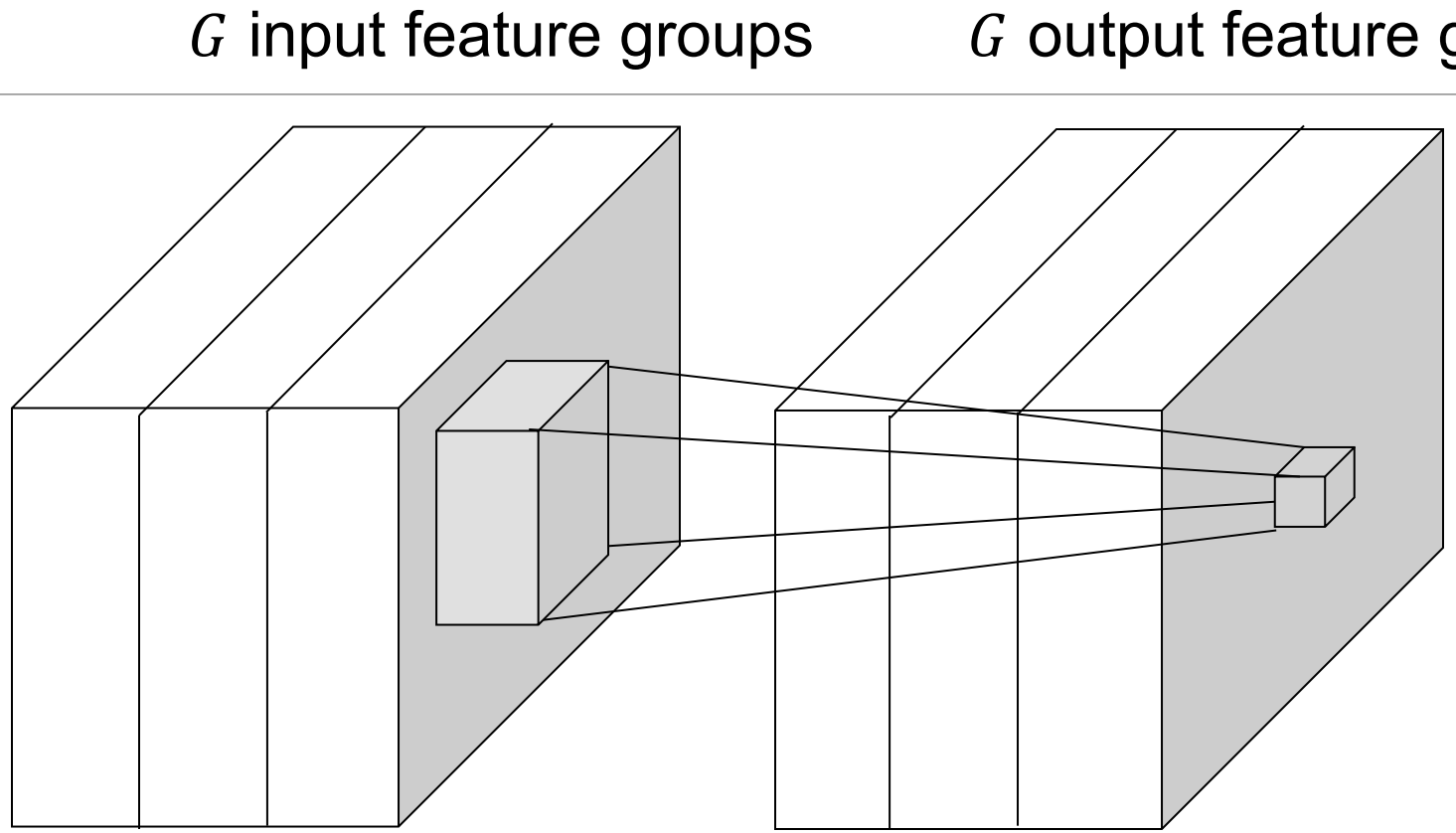

Depthwise Convolutional Filters [1]




Slide credit: Prof. Svetlana Lazebnik

[1] A. Howard et al., MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications, arXiv 2017

# More generally: Group-wise convolutions



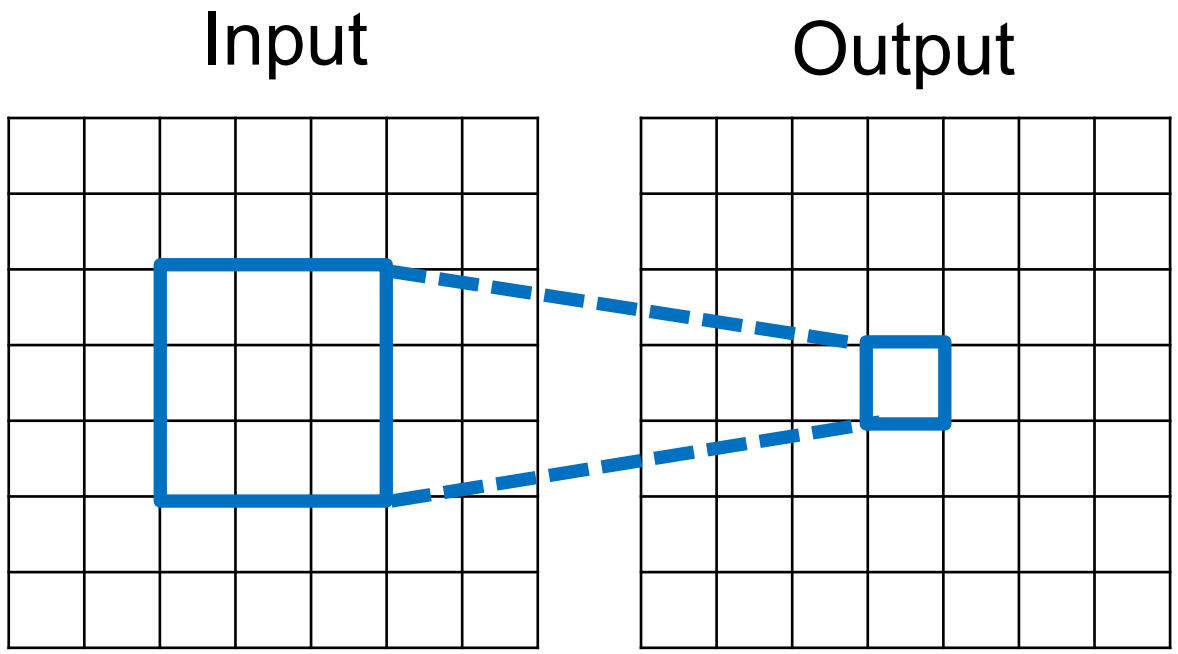

- Split up the $K$ feature maps into $G$ groups, perform convolutions within each group separately, concatenate the results

Slide credit: Prof. Svetlana Lazebnik

# Receptive field

3x3 convolutions, stride 1

The *receptive field* of a unit is the region of the input feature map whose values contribute to the response of that unit (either in the previous layer or in the initial image)

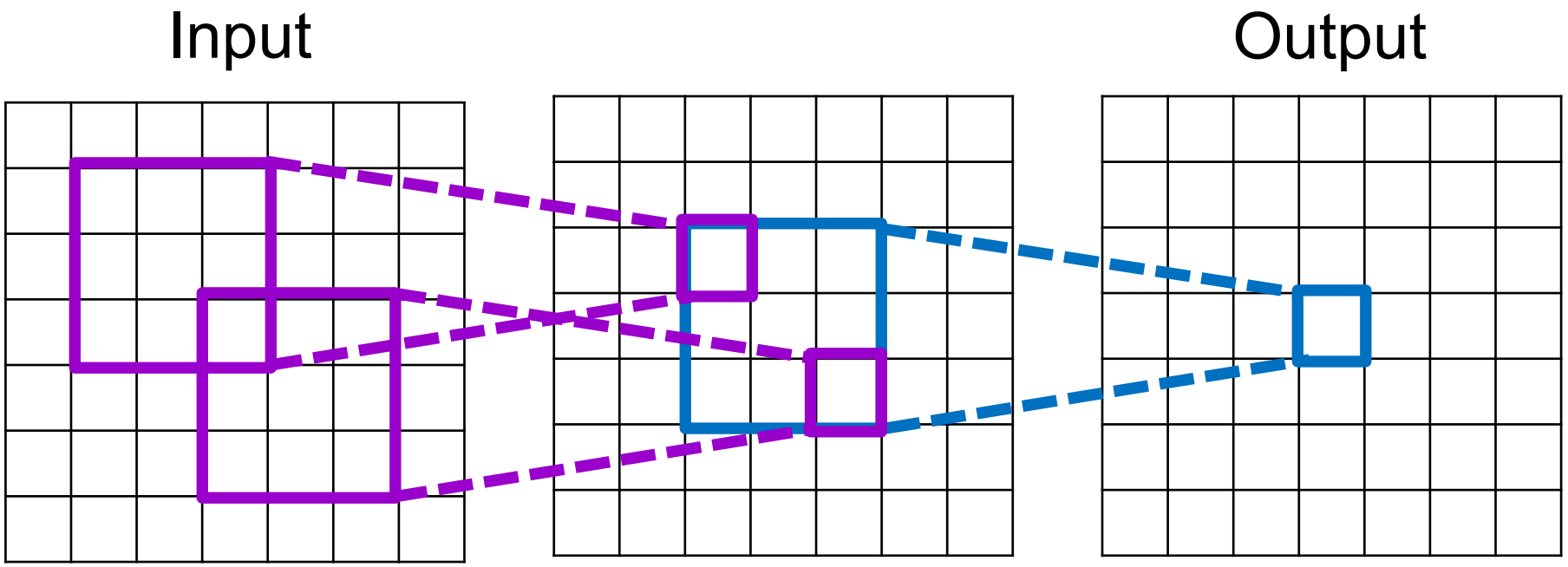



Receptive field size: 3
(each dimension)

Slide credit: Prof. Svetlana Lazebnik

# Receptive field

3x3 convolutions, stride 1

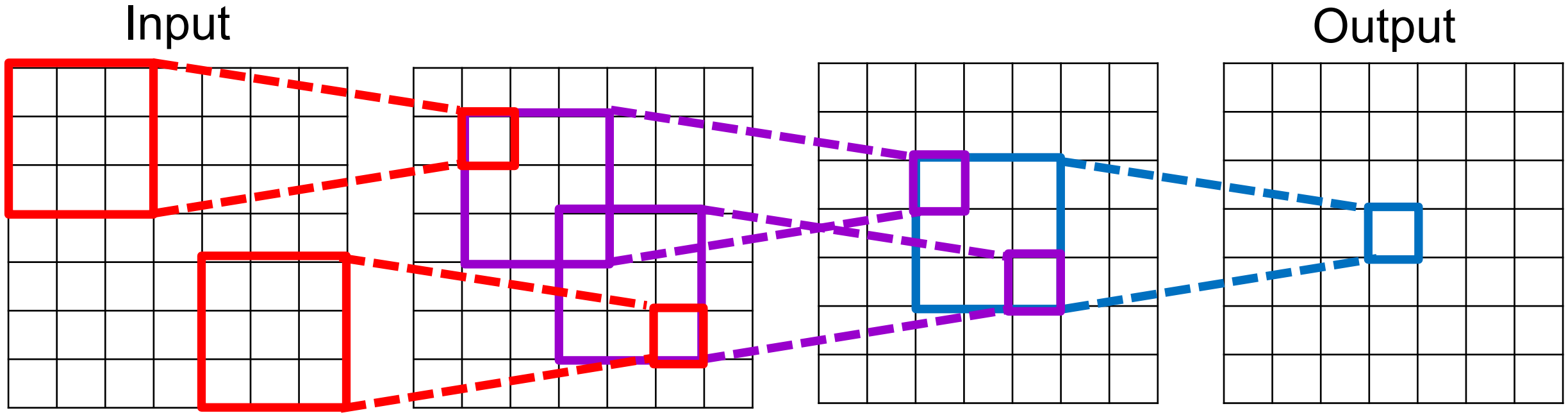



Receptive field size: 5

Slide credit: Prof. Svetlana Lazebnik

# Receptive field

3x3 convolutions, stride 1




Each successive convolution adds $F - 1$ to the receptive field size
With $L$ layers the receptive field size is $1 + L * (F - 1)$

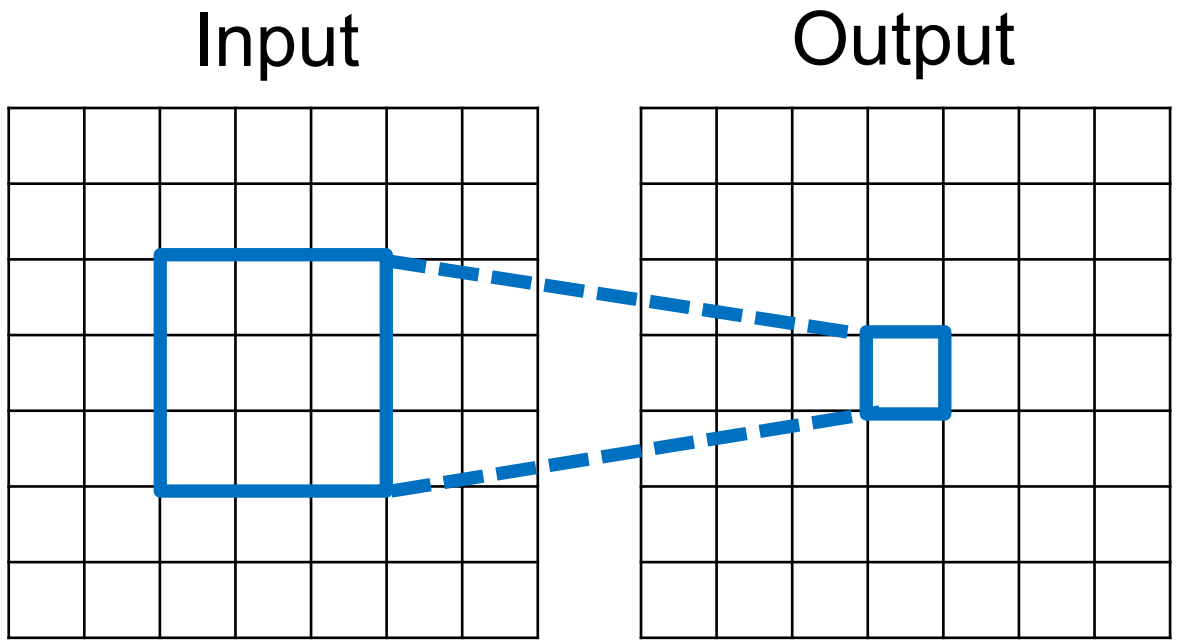
Receptive field size: 7

- If you have 3 layers, each with a 3×3 filter,
  Receptive field size: $1 + 3 * (3 - 1) = 7$

Slide credit: Prof. Svetlana Lazebnik

# Receptive field

3x3 convolutions, stride 2

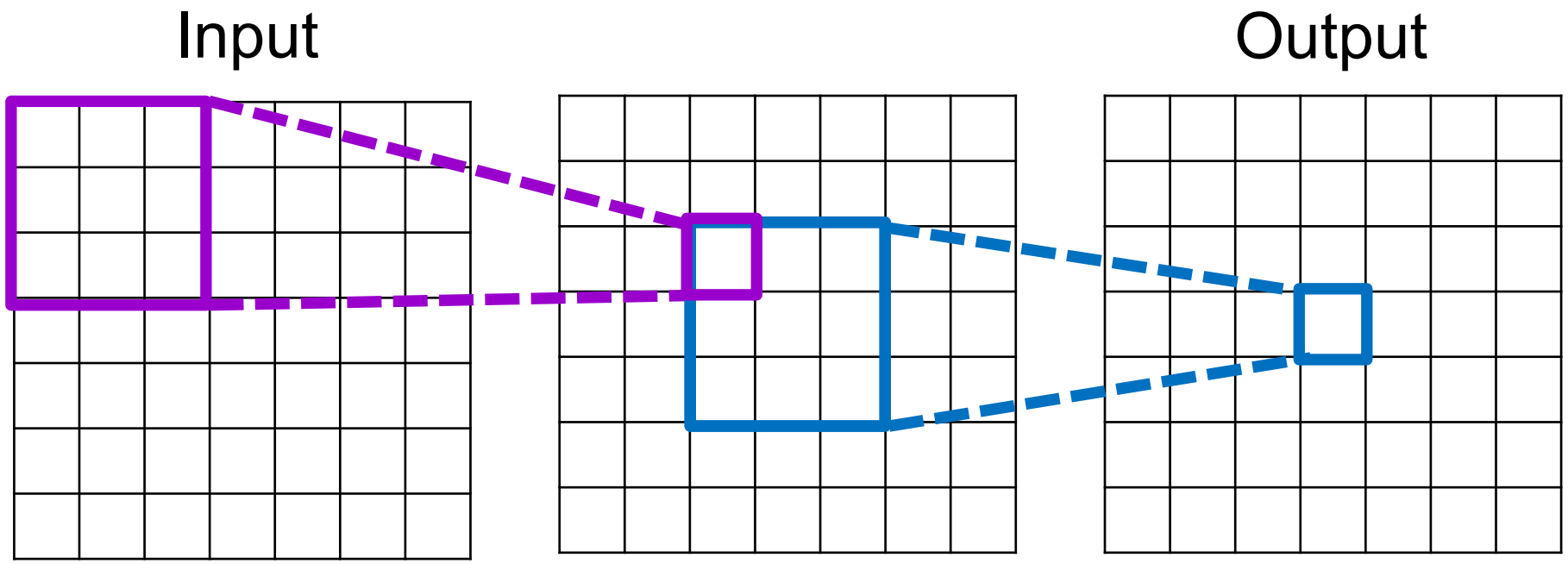



Receptive field size: 3

Slide credit: Prof. Svetlana Lazebnik

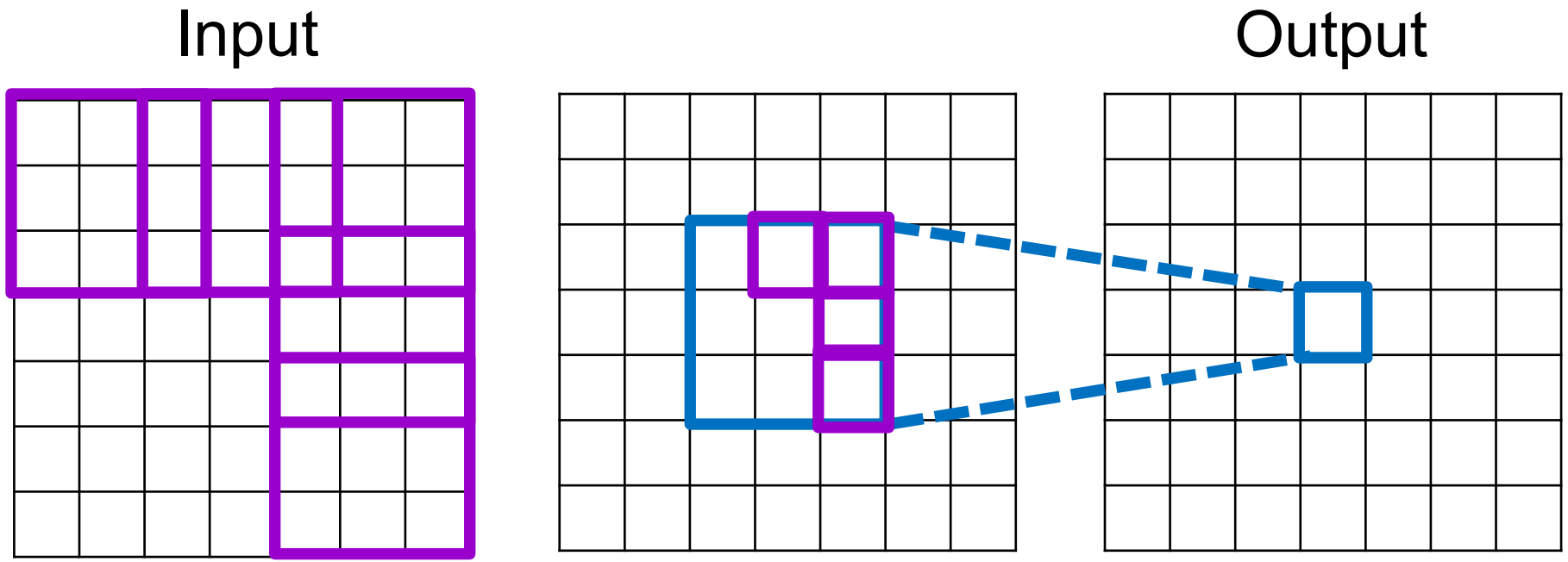
# Receptive field

3x3 convolutions, stride 2



# Receptive field

3x3 convolutions, stride 2



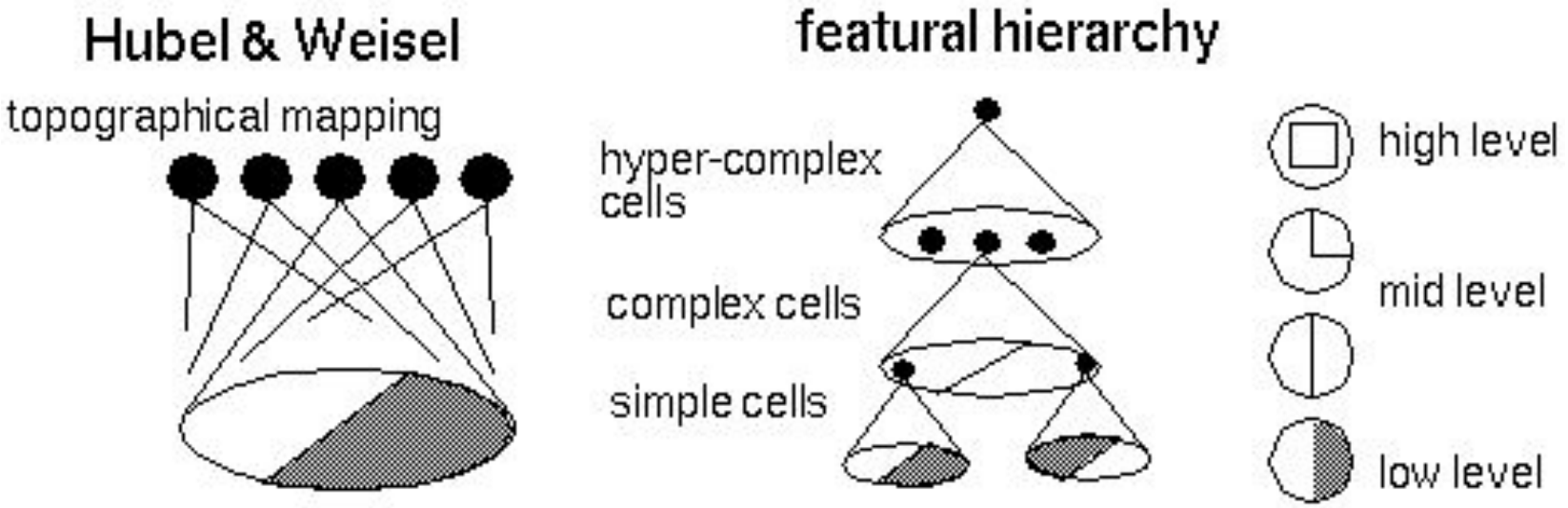

With a stride of 2, receptive field size is given by $2^{L+1} - 1$, i.e., it grows exponentially (though spatial resolution decreases exponentially)

Receptive field size: 7

Slide credit: Prof. Svetlana Lazebnik

# Backstory

- Biological inspiration: D. Hubel and T. Wiesel (1959, 1962, Nobel Prize 1981)
  - Visual cortex consists of a hierarchy of *simple*, *complex*, and *hyper-complex* cells

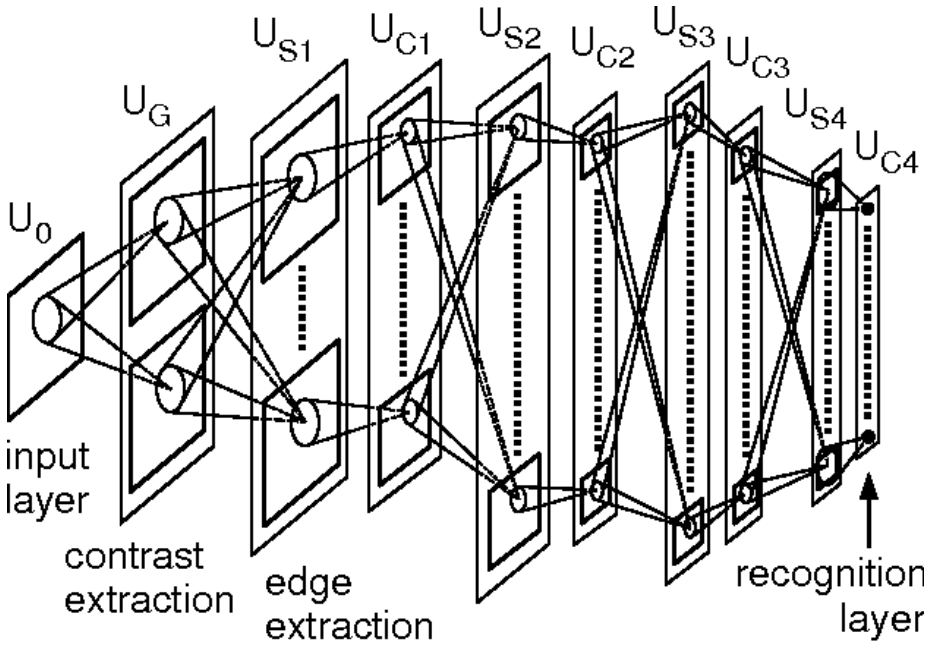



Slide credit: Prof. Svetlana Lazebnik

Source

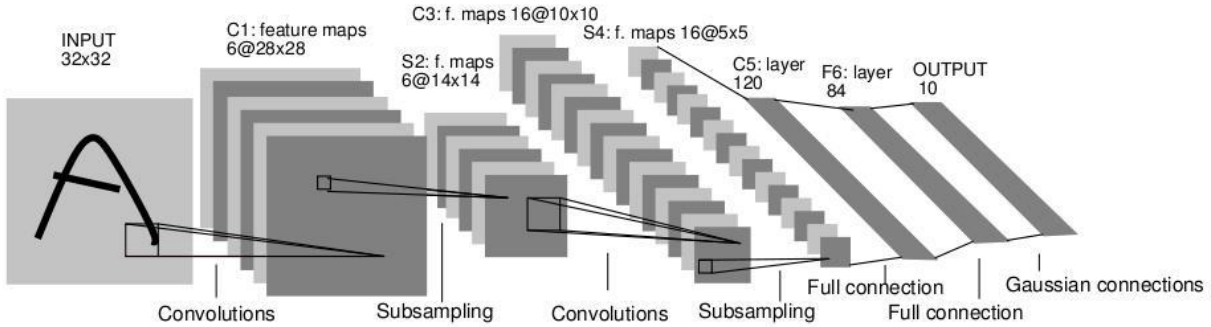
# Backstory

## Neocognitron




K. Fukishima. Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position. Biological Cybernetics, 1980

## LeNet-5




Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, Gradient-based learning applied to document recognition, Proc. IEEE 86(11): 2278–2324, 1998

Slide credit: Prof. Svetlana Lazebnik

# Example

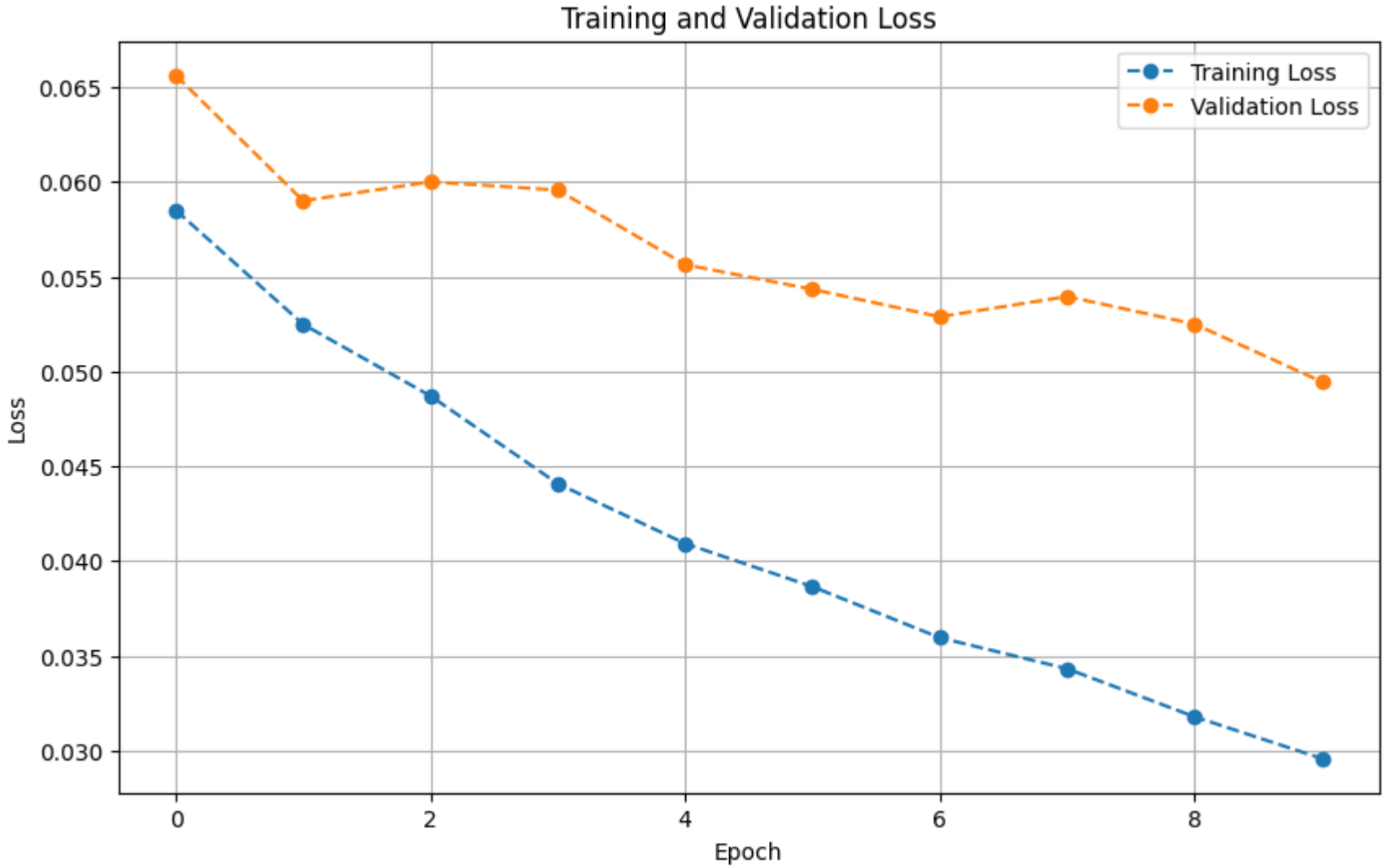
```
# Define the CNN model
model = keras.Sequential([
    # First Convolutional Layer with 32 filters and a 3x3 kernel
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    # MaxPooling layer to downsample the feature maps
    layers.MaxPooling2D((2, 2)),        # Second Convolutional Layer with 64 filters and a 3x3 kernel
    layers.Conv2D(64, (3, 3), activation='relu'),
    # MaxPooling layer to downsample the feature maps
    layers.MaxPooling2D((2, 2)),
    # Flatten the 2D feature maps to a 1D array
    layers.Flatten(),
    # Add a dense (fully connected) layer with 128 units and ReLU activation
    layers.Dense(128, activation='relu'),
    # Add another dense layer with 64 units and ReLU activation
    layers.Dense(64, activation='relu'),
    # Add the output layer with 10 units (one for each class) and softmax activation
    layers.Dense(10, activation='softmax')
])
```

```
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d_6 (Conv2D)            (None, 26, 26, 32)        320
_________________________________________________________________
max_pooling2d_6 (MaxPooling2 (None, 13, 13, 32)        0
_________________________________________________________________
conv2d_7 (Conv2D)            (None, 11, 11, 64)        18496
_________________________________________________________________
max_pooling2d_7 (MaxPooling2 (None, 5, 5, 64)          0
_________________________________________________________________
flatten_3 (Flatten)          (None, 1600)              0
_________________________________________________________________
dense_9 (Dense)              (None, 128)               204928
_________________________________________________________________
dense_10 (Dense)             (None, 64)                8256
_________________________________________________________________
dense_11 (Dense)             (None, 10)                650
=================================================================
Total params: 232,650
Trainable params: 232,650
Non-trainable params: 0
```

The convolution layers account for the majority of floating-point operations. Most of the memory utilizes by convolution layers.

The majority of parameters reside within the fully-connected dense layers.

# Example




```
Test Loss: 0.042
Test Accuracy: 0.987
```

# More examples

- Pytorch training a classifier
- Pytorch build a model
- keras image classification from scratch
- keras image classification via fine-tuning with EfficientNet

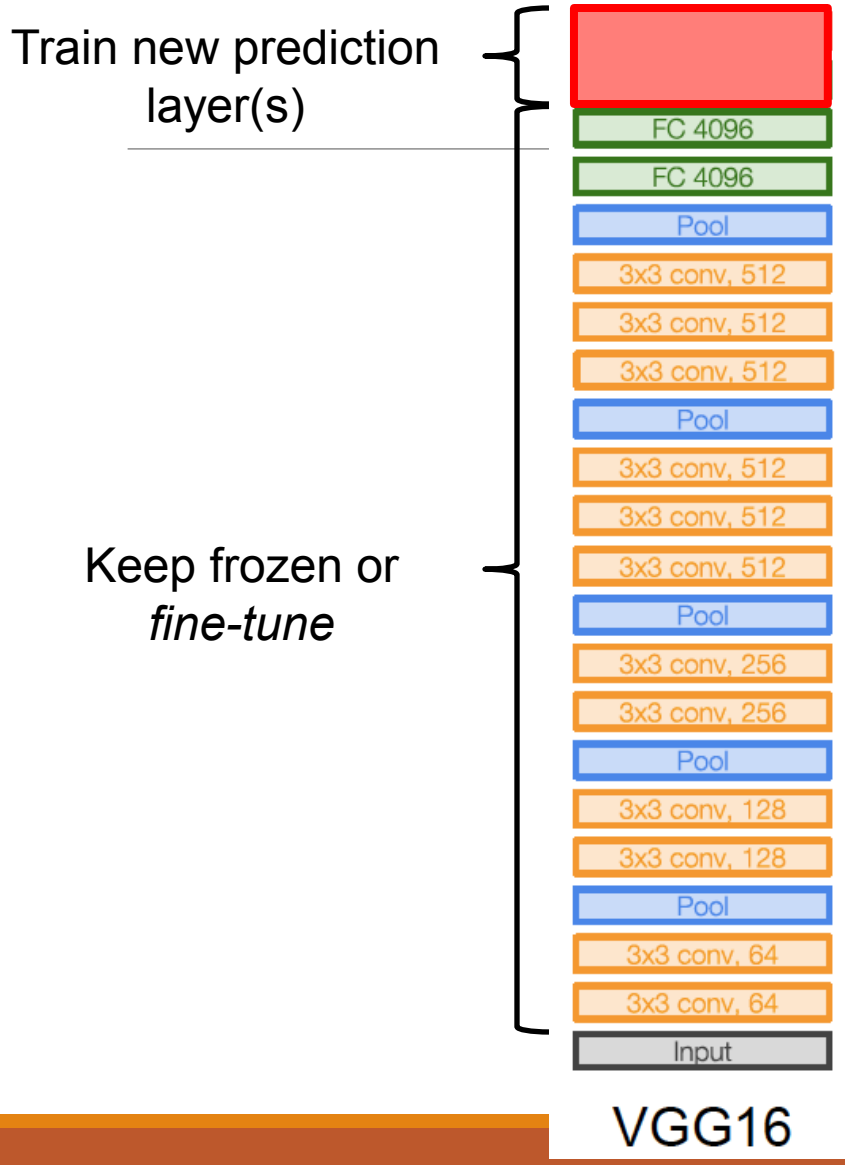
# How to use a pre-trained network for a new task?




- Strategy 1: Use as feature extractor

Slide credit: Prof. Svetlana Lazebnik

# How to use a pre-trained network for a new task?




- Strategy 2: Transfer learning

https://keras.io/examples/vision/image_classification_efficientnet_fine_tuning/





# Thank You

# Q & A